

Artificial Intelligence LAB

(CIE-374P)

Faculty Name : Mr. Saurabh Rastogi

Student Name : Yash Pandey

Roll No : 12214802722 (AIML II-B)

Semester : 6th Semester



Maharaja Agrasen Institute of Technology, PSP Area,

Sector – 22, Rohini, New Delhi - 110086



MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

VISION

To attain global excellence through education, innovation, research, and work ethics with the commitment to serve humanity digitize educational resources and process for enhanced teaching and effective learning.

MISSION

- M1. To promote diversification by adopting advancement in science, technology, management, and allied discipline through continuous learning
- M2. To foster moral values in students and equip them for developing sustainable solutions to serve both national and global needs in society and industry.
- M3. To digitize educational resources and process for enhanced teaching and effective learning.
- M4. To cultivate an environment supporting incubation, product development, technology transfer, capacity building and entrepreneurship.
- M5. To encourage faculty-student networking with alumni, industry, institutions, and other stakeholders for collective engagement.



Department of Computer Science & Engineering

VISION

"To attain global excellence through education, innovation, research, and work ethics in the field of Computer Science and engineering with the commitment to serve humanity."

MISSION

M1 To lead in the advancement of computer science and engineering through internationally recognized research and education.

M2 To prepare students for full and ethical participation in a diverse society and encourage lifelong learning.

M3 To foster development of problem solving and communication skills as an integral component of the profession.

M4 To impart knowledge, skills and cultivate an environment supporting incubation, product development, technology transfer, capacity building and entrepreneurship in the field of computer science and engineering.

M5 To encourage faculty, student's networking with alumni, industry, institutions, and other stakeholders for collective engagement



VISION

"To attain global excellence through education, innovation, research, and work ethics in the field of Computer Science and engineering with the commitment to serve humanity."

MISSION

M1 To lead in the advancement of computer science and engineering through internationally recognized research and education.

M2 To prepare students for full and ethical participation in a diverse society and encourage lifelong learning.

M3 To foster development of problem solving and communication skills as an integral component of the profession.

M4 To impart knowledge, skills and cultivate an environment supporting incubation, product development, technology transfer, capacity building and entrepreneurship in the field of computer science and engineering.

M5 To encourage faculty, student's networking with alumni, industry, institutions, and other stakeholders for collective engagement.



POs, PSOs & PEOs

Program Outcomes (POs)

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change

Program Specific Outcomes (PSOs)

PSO1: Able to explore and apply emerging technologies in computer science and engineering such as Artificial Intelligence, Machine Learning, Data Science, etc.

PSO2: Able to independently and collaboratively design, develop and evaluate innovative solutions to existing problems, addressing the needs of industry and society.

PSO3: Able to pursue advanced studies, conduct research and development, and cultivate entrepreneurship skills in the modern computing environment.

Program Educational Objectives (PEOs)

PEO1: Graduates will work with the top institutions and researchers, dedicating themselves to lifelong learning and social responsibility. **(M1, M2)**

PEO2: Graduates will exhibit outstanding communication skills and the capacity to collaborate effectively within diverse teams. **(M3)**

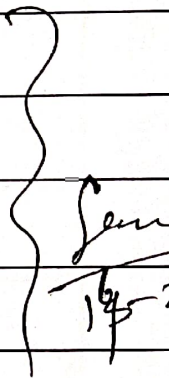
PEO3: Graduates cultivating skills in computer science and engineering contribute to driving innovation, entrepreneurship, and economic growth. **(M4)**

PEO4: Graduates network with stakeholders to contribute to the growth of the department. **(M5)**

Rubrics for Lab Assessment:

Rubrics		10 Marks			POs and PSOs Covered	
		0 Marks	1 Marks	2 Marks	PO	PSO
R1	Is able to identify and define the objective of the given problem?	No	Partially	Completely	PO1, PO2	PSO1, PSO2
R2	Is proposed design/procedure/algorithm solves the problem?	No	Partially	Completely	PO1,PO2, PO3	PSO1, PSO2
R3	Has the understanding of the tool/programming language to implement the proposed solution?	No	Partially	Completely	PO1,PO3, PO5	PSO1, PSO2
R4	Are the result(s) verified using sufficient test data to support the conclusions?	No	Partially	Completely	PO2,PO4, PO5	PSO2
R5	Individuality of submission?	No	Partially	Completely	PO8, PO12	PSO1, PSO3

LAB INDEX

S.No	Experiment	R1	R2	R3	R4	R5	Total Marks (10)	Remarks	Faculty Signature
		Is able to identify and define the objective of the given problem?	Is proposed design /procedure /algorithm solves the problem?	Has the understanding of the tool/programming language to implement the proposed solution?	Are the result(s) verified using sufficient test data to support the conclusions?	Individuality of submission?			
		2 Marks	2 Marks	2 Marks	2 Marks	2 Marks			
1	Study of Prolog								
2	Simple facts for Statements								
3	Write Predicate To Convert °C → F								
4	WAP To Implement Breadth First Search								
5	WAP To Implement Water - Jug Problem								
6	WAP To Implement Tic - Toe - Toe								
7	WAP To Remove Pronunciations								
8	WAP To Sort The Sentences								
9	WAP for Hangman Game In Prolog								
10	WAP for Hangman Game In Python								
11	WAP To Parse Text file with NLTK								
12	WAP To Implement Stemming NLTK								

LAB INDEX

[illegible]

LAB INDEX

[illegible]

Experiment-1

Aim:- Study of PROLOG.

PROLOG stands for Programming in Logic — an idea that emerged in the early 1970's to use logic as programming language. It is a programming language that's well-suited for developing logic-based artificial intelligence applications. The early developers of this idea included Robert Kowalski at Edinburgh (on the theoretical side), Marriten van Emden at Edinburgh (experimental demonstration) and Alan Colmerauer at Marseilles (implementation). David D.H. Warren's efficient implementation at Edinburgh in the mid -1970's greatly contributed to the popularity of PROLOG.

It is a declarative programming language designed for developing logic-based AI applications. Developers can set rules and facts around a problem, and then Prolog's interpreter will use that information to automatically infer solutions.

It is a programming language centred around a small set of basic mechanisms, including pattern matching, tree-based data structuring and automatic backtracking. This Small set constitutes a surprisingly powerful and flexible programming framework. PROLOG is especially well suited for problems that involve objects-in-particular, structured objects-and relations between them.

Facts, Rules, Queries, Predicates and Clauses:

A Prolog program consists of a number of clauses. Each clause is either a fact or a rule. After a Prolog program is loaded (or consulted) in a Prolog interpreter, users can submit goals or queries, and the Prolog interpreter will give results (answers) according to the facts and rules.

Facts: A fact must start with a predicate (which is an atom) and end with a full stop. The predicate may be followed by one or more arguments which are enclosed by parentheses. The arguments can be atoms (in this case, these atoms are treated as constants), numbers, variables or lists. Arguments are separated by commas. For example: man(john).

If we consider the arguments in a fact to be objects, then the predicate of the fact describes a property of the objects. In a Prolog program, a presence of a fact indicates a statement that is true. An absence of a fact indicates a statement that is not true.

Rules: A rule can be viewed as an extension of a fact with added conditions that also have to be satisfied for it to be true. It consists of two parts. The first part is similar to a fact (a predicate with arguments). The second part consists of other clauses (facts or rules which are separated by commas) which must all be true for the rule itself to

be true. These two parts are separated by ":-". You may interpret this operator as "if" in English. `not(X,Y) :- man(X), woman(Y).`

Queries: A query is a statement starting with a predicate and followed by its arguments, some of which are variables. Similar to goals, the predicate of a valid query must have appeared in at least one fact or rule in the consulted program, and the number of arguments in the query must be the same as that appears in the consulted program.

The purpose of submitting a query is to find values to substitute into the variables in the query such that the query is satisfied. This is similar to asking a question, asking for "what values will make my statement true". For example: `not(john, mary)?`

Predicates:

- A predicate is a function or relation that takes some arguments and returns a boolean value (true or false). Predicates represent properties or relations in a logical system.
- Example: In the predicate `cat(mittens)`, "cat" is the predicate, and "mittens" is the argument.

Clauses:

- A clause in logic typically refers to a rule or a fact. In Prolog, a clause can be a fact or a rule. A clause is composed of a head and a body.

1. Fact: `cat(mittens).`
2. Rule: `mammal(X) :- cat(X).`

- Clauses are used to define relationships between different facts in a knowledge base.

Features of Prolog:

- **Declarative Nature:** Focuses on *what* needs to be solved rather than *how* to solve it.
- **Logical Syntax:** Uses facts, rules, and queries to represent relationships and logic.
- **Backtracking:** Automatically searches for all possible solutions by backtracking when it hits a dead-end.
- **Unification:** Matches terms in queries with facts and rules by finding substitutions for variables.
- **Pattern Matching:** Compares and matches terms in the query with those in the knowledge base.
- **Non-determinism:** Can return multiple solutions to a problem by exploring different possible answers.

- **Recursion:** Commonly used to process data and define relationships, especially with lists or trees.
- **Built-in Predicates:** Provides pre-defined predicates for tasks like arithmetic, list manipulation, and more.
- **Dynamic Database:** Allows the addition and removal of facts and rules during program execution (assert/retract).
- **Constraint Logic Programming (CLP):** Extends Prolog for solving problems with constraints, such as optimization or scheduling.
- **Natural Language Processing (NLP):** Well-suited for parsing and interpreting languages, often used in AI and linguistics.
- **Built-in Lists and Data Structures:** Native support for lists and other common data structures for handling collections of terms.
- **No Side Effects:** Purely declarative, with no hidden state or side effects during computation.
- **Interactivity:** Supports interactive development where users define and query facts, rules, and conditions in real-time.

SYMBOLIC LANGUAGE:

PROLOG is a programming language for symbolic, non-numeric computation. It is especially well suited for solving problems that involve objects and relations between objects. For example, it is an easy exercise in PROLOG to express spatial relationship between objects, such as the blue sphere is behind the green one. It is also easy to state a more general rule: if object X is closer to the observer than object Y, and object Y is closer than Z, then X must be closer than Z. PROLOG can reason about the spatial relationships and their consistency with respect to the general rule. Features like this make PROLOG a powerful language for Artificial LanguageA1,) and non-numerical programming. There are well-known examples of symbolic computation whose implementation in other standard languages took tens of pages of indigestible code, when the same algorithms were implemented in PROLOG, the result was a crystal-clear program easily fitting on one page.

1. It uses **symbols** to represent facts, rules, and queries in its knowledge base.
2. Symbols in Prolog include constants, variables, predicates, and operators.
3. Logical relationships are expressed through symbolic rules, using operators like :- (if) and , (and).
4. Prolog's symbolic nature makes it ideal for applications like **artificial intelligence** and **knowledge representation**.
5. The combination of symbols and logical inference allows Prolog to solve complex problems through reasoning.

SYMBOLS USED IN PROLOG:

Symbol	Name	Usage
.	Full stop	Used to terminate clauses or queries.
:-	Rule operator	Denotes "if" in rules, separating the head from the body.
,	Comma	Represents a logical AND in Prolog queries or rule bodies.
;	Semicolon	Represents a logical OR in queries or rules.
=	Equality operator	Used to test if two terms are the same.
\=	Inequality operator	Tests if two terms are not the same.
==	Strict equality operator	Tests if two terms are identical without variable bindings.
\==	Strict inequality	Tests if two terms are not identical without variable bindings.
is	Arithmetic evaluation	Evaluates an arithmetic expression and unifies it with a variable.
=<	Less than or equal to	Used for arithmetic comparisons.
<	Less than	Used for arithmetic comparisons.
>=	Greater than or equal to	Used for arithmetic comparisons.
>	Greater than	Used for arithmetic comparisons.

META PROGRAMMING:

- **Program as Data:** Prolog treats code as data, allowing the program to manipulate or inspect other parts of itself during execution.
- **Dynamic Predicates:** Prolog allows predicates to be asserted (added) or retracted (removed) at runtime, enabling dynamic modifications of the knowledge base.
- **Term Construction and Unification:** Prolog allows the dynamic construction of terms and the unification of terms, enabling flexible manipulation of predicates and goals.
- **The =.. Operator (Univ):** This operator allows you to convert a term into a list of its components or vice versa, providing a way to dynamically generate or inspect predicates.
- **Creating and Modifying Rules Dynamically:** Prolog allows the creation or modification of rules at runtime, enabling the program to adapt and change based on new information.

- **Reflection:** Prolog can introspect its own code to retrieve or manipulate predicates, facts, and clauses, enabling self-examination and modification of the program.
- **Goal Construction:** Prolog supports dynamic creation of goals at runtime, enabling flexible query generation and execution.
- **Higher-Order Predicates:** Prolog allows predicates to accept other predicates as arguments, supporting higher-order logic and enabling more abstract programming.
- **Meta-Interpretation:** Prolog programs can interpret or execute other Prolog programs, allowing for custom reasoning or translation systems.

EXPERT SYSTEM IN DESIGNING PROLOG:

1. Knowledge Base:

- The knowledge base consists of facts and rules that represent the expert knowledge in a specific domain. This knowledge is encoded in Prolog as facts (assertions about the world) and rules (logical relationships that define how knowledge is derived).
- **Example:** Facts could describe relationships, like `parent(john, mary)` (John is Mary's parent), while rules could describe derived knowledge, like `grandparent(X, Y) :- parent(X, Z), parent(Z, Y)` (X is a grandparent of Y if X is a parent of Z and Z is a parent of Y).

2. Inference Engine:

- The inference engine is the component that applies logical reasoning to the knowledge base to draw conclusions. In Prolog, this is done using backtracking and unification. It attempts to match queries with the knowledge base and deduces new information by applying rules.
- It processes queries by finding relevant facts and applying rules to infer answers.

3. User Interface:

- A user interface allows the system to interact with the user, typically asking questions and interpreting their answers to form new facts. This could involve interactive prompts or structured inputs.
- Prolog's interactive nature, where you can query the system and receive answers, fits well as an interface for expert systems.

4. Explanation Facility:

- An explanation facility helps the system explain its reasoning process to the user. In Prolog, this could involve explaining how a conclusion was reached by showing the chain of facts and rules that led to the result.
- This can be implemented by tracing the rules and facts used to derive a particular conclusion.

5. Working Memory:

- The working memory stores facts that are known to be true at a given moment in time. This memory is updated as new facts are discovered during the reasoning process.

INSTALLATION STEPS:

Step 1: Visit swi-prolog.org website using any web browser.

Step 2: Click on Download which is adjacent to Home, dropdown list will appear then click on SWI-Prolog.

Step 3: New webpage will open, click on Stable release.

Step 4: After clicking on stable release new webpage will open which will contain stable versions of prolog for different platforms. Under binaries there are two stable releases for windows, first is SWI-Prolog 8.4.0-1 for Microsoft Windows (64 bit) and the other is SWI-Prolog 8.4.0-1 for Microsoft Windows (32 bit). Click on the one as per your system configuration. Let's take the one for 64-bit operating system.

Step 5. After clicking on SWI-Prolog 8.4.0-1 for Microsoft Windows (64 bit), a new webpage will open, check on I understand checkbox to make the download link active. Then click on the download link, downloading of the executable file will start shortly. It is a small 11.9 MB file that will hardly take a minute.

Step 6: Now check for the executable file in downloads in your system and run it.

Step 7: It will prompt confirmation to make changes to your system. Click on Yes.

Step 8: Setup screen will appear, click on Next.

Step 9: The next screen will be of License Agreement, click on I Agree.

Step 10: After it there will be screen of installing options so check the box for Add swipl to the system path for all users, and also check the box for create a desktop icon and then click on the Next button.

Step 11: The next screen will be of installing location so choose the drive which will have sufficient memory space for installation. It needed only a memory space of 50 MB.

Step 12: Next screen will be of choosing Start menu folder so don't do anything just click on Next Button.

Step 13: This last screen is of choosing components, all components are already marked so don't change anything just click on Install button.

Step 14: After this installation process will start and will hardly take a minute to complete the installation.

Step 15: Click on Finish after the installation process is complete.

Step 16: SWI Prolog is successfully installed on the system and an icon is created on the desktop.

Step 17: Run the software and see the interface.

Q: Program to print Hello World.

Code- | `?- write('Hello World!'), nl.`

Output:

```
| ?- write('Hello World!'), nl.  
Hello World!
```

Program – 1

Aim: Study of Prolog

Introduction to Prolog

Prolog (short for "Programming in Logic") is a declarative programming language, meaning it focuses on *what* to solve rather than *how* to solve it. Developed in the early 1970s by Alain Colmerauer and Philippe Roussel, Prolog is based on formal logic, specifically predicate logic. It is widely used in artificial intelligence (AI) applications, especially in areas like natural language processing, expert systems, and knowledge representation.

Unlike languages such as C, Java, or Python, where the programmer specifies a sequence of steps to execute, Prolog allows the programmer to describe logical relationships between data. Prolog then figures out the steps needed to derive a solution, making it an excellent fit for solving problems that involve complex relationships, like reasoning and decision-making.

Facts in Prolog

Facts are the simplest building blocks in Prolog. A fact is a statement about the world, which is unconditionally true. In other words, it asserts that something exists or a relationship holds without needing to be proved.

Example of Facts:

```
father(john, mary).  
mother(mary, alice).
```

These are basic facts, stating:

- John is the father of Mary.
- Mary is the mother of Alice.

In Prolog, facts are written as `predicate(arguments) .`, where the `predicate` represents the relation, and `arguments` are the entities involved.

- **Father/mother** are predicates.
- `john`, `mary`, and `alice` are constants (or atomic values).

When a fact is included in the Prolog program, Prolog assumes it to be true, and it can be used in rules and queries for further inferences.

Rules in Prolog

Rules in Prolog are used to define relationships that hold true under certain conditions. A rule can be thought of as a conditional statement, similar to "if-then" statements in other

languages. The body of a rule specifies the conditions that must be true for the rule to be applied.

A rule is made up of two parts:

1. **Head:** The conclusion or outcome of the rule.
2. **Body:** The conditions that must be satisfied for the head to be true.

Example of a Rule:

```
grandfather(X, Y) :- father(X, Z), father(Z, Y).
```

This rule reads as "X is a grandfather of Y if X is a father of Z and Z is a father of Y." The symbol `:-` is read as "if." So, Prolog will deduce the relationship `grandfather(X, Y)` only if the conditions `father(X, Z)` and `father(Z, Y)` are both true.

The body of the rule contains two predicates, `father(X, Z)` and `father(Z, Y)`. Prolog will attempt to satisfy both conditions in sequence when trying to determine if the rule applies.

Predicates in Prolog

A **predicate** is a symbol that represents a relation between terms. Predicates can be used in facts and rules. They are the essence of defining relationships in Prolog.

In the rule:

```
grandfather(X, Y) :- father(X, Z), father(Z, Y).
```

`grandfather/2` and `father/2` are predicates. The `/2` indicates that each predicate has two arguments, which is the arity of the predicate. A predicate can have any number of arguments.

- In the fact:
 - `father(john, mary).`
`father/2` is the predicate, and `john` and `mary` are the arguments.

In Prolog, predicates are used to express relationships. They can represent complex relations like "is a parent of," "is a sibling of," "is a friend of," and so on. These relationships form the core of Prolog's logic system.

Clauses in Prolog

A **clause** in Prolog is either a fact or a rule. It is a fundamental part of any Prolog program and provides a way to express logical relationships.

A Prolog program consists entirely of clauses, which can be:

1. **Facts:** Unconditional truths.
2. **Rules:** Conditional truths (used for inference).

Example of a Clause:

```
father(john, mary). % Fact
grandfather(X, Y) :- father(X, Z), father(Z, Y). % Rule
```

A **fact** simply declares something true, while a **rule** expresses a relationship between facts that Prolog can use to deduce new facts. A Prolog program consists of a collection

of such clauses. When querying Prolog, it will attempt to match the query with facts and use rules to infer additional facts.

Features of Prolog

Prolog has several distinct features that make it particularly suited for specific kinds of tasks, especially in AI and logic-based applications:

1. **Declarative Nature:**
 - In Prolog, you describe *what* the problem is and let the system figure out *how* to solve it. This contrasts with procedural languages, where you explicitly define the sequence of operations.
 - This makes Prolog particularly suited for knowledge representation, reasoning, and problem-solving tasks.
2. **Automatic Backtracking:**
 - Prolog automatically searches for solutions by trying different paths and backtracking when it reaches a dead end. This allows Prolog to explore multiple possible solutions without the programmer needing to manually implement backtracking logic.
3. **Pattern Matching:**
 - Prolog uses pattern matching to check if a goal (query) matches with facts or rules in the knowledge base. It tries to unify the terms in the query with terms in the facts or rules.
4. **Recursion:**
 - Prolog's approach is heavily recursive, which means that problems are often solved by breaking them down into smaller, self-similar subproblems. Prolog's rules and queries often define a solution in terms of simpler instances of the same problem.
5. **Unification:**
 - Unification is a central concept in Prolog. It refers to the process of making two terms identical by finding an appropriate substitution for their variables. This allows Prolog to match queries with facts and rules dynamically.
6. **Logical Variables:**
 - Prolog uses variables that can be instantiated during the execution of the program. When Prolog tries to satisfy a query, it will substitute values for variables to make the query true.

Symbolic Language in Prolog

Prolog is often called a **symbolic language** because it deals with symbolic terms rather than raw data. In contrast to numeric computation or simple data manipulation, Prolog focuses on relationships between symbolic terms.

- **Atoms:** These are basic symbols in Prolog. An atom can be a simple constant like `john` or a more complex structure like `'hello world'`.

- **Variables:** Represent unknown values that Prolog will attempt to assign values to during execution. Variables are usually written in uppercase, like `X`, `Y`, or `Person`.
- **Predicates:** Symbolic names for relations between terms, such as `father/2`, `likes/2`, etc.
- **Terms:** A term in Prolog can be an atom, a variable, or a compound term (a structure). Compound terms consist of a functor and a list of arguments, e.g., `father(john, mary)` is a compound term.

By using these symbols, Prolog allows for logical reasoning based on abstract relationships and structures, making it suitable for tasks that require symbol manipulation, such as language processing or expert systems.

Symbols in Prolog

Prolog uses a rich set of symbols to represent different types of data and relationships:

- **Atoms:** Basic symbols, like `alice`, `john`, `new_york`, and so on.
- **Variables:** Represented by identifiers that start with uppercase letters, such as `X`, `Person`, or `Temp`.
- **Predicates:** Represent relationships, like `father/2`, `likes/2`, `grandfather/2`.
- **Functors:** These are used to define compound terms, such as `father(john, mary)`. A functor is the name of the predicate that takes arguments.
- **Constants:** Simple values like `john`, `pizza`, etc.
- **Compound Terms:** Terms made of a functor and its arguments, such as `father(john, mary)`.

Meta-Programming in Prolog

Meta-programming in Prolog is the practice of writing programs that treat other programs as data. This allows Prolog to generate or modify rules and facts at runtime, providing powerful capabilities for dynamic reasoning and reflection.

Example of Meta-Programming:

```
assert(father(john, mary)).
```

This adds a fact `father(john, mary)` to the knowledge base during execution.

Similarly, Prolog has predicates like `retract/1`, `assertz/1`, and `asserta/1` for manipulating the knowledge base.

Meta-programming is used in applications like:

- **Dynamic knowledge base updates:** Adding or removing facts during program execution.
- **Creating interpreters:** Building systems that interpret or execute other Prolog programs.

Expert Systems in Decision Prolog

An **expert system** is a software application that emulates the decision-making ability of a human expert. Prolog is particularly well-suited for developing expert systems because it

can easily represent complex relationships and make logical inferences based on a set of rules.

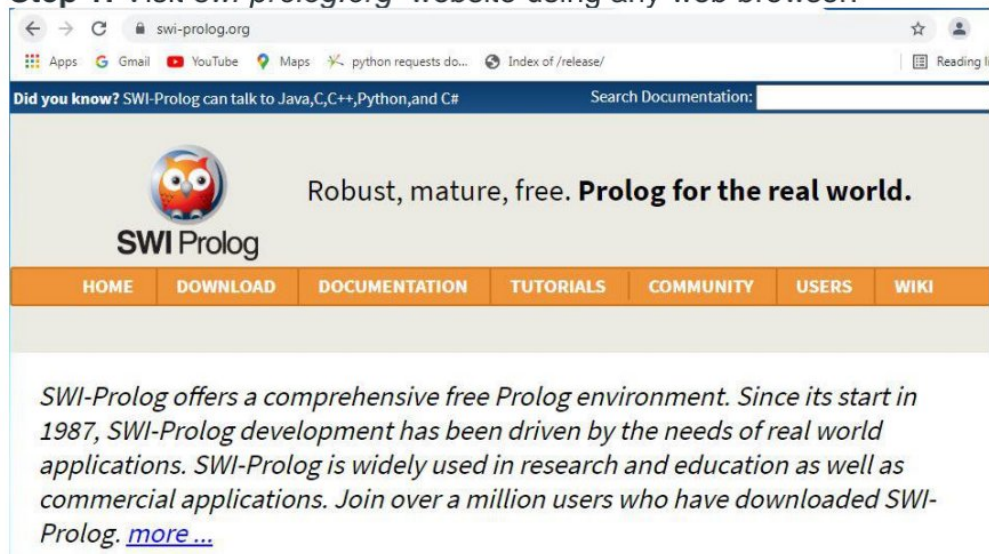
For instance, an expert system for diagnosing medical conditions might include rules like:

```
fever(yes) :- temperature(> 100) .  
cough(yes) :- throat_infection(yes) .
```

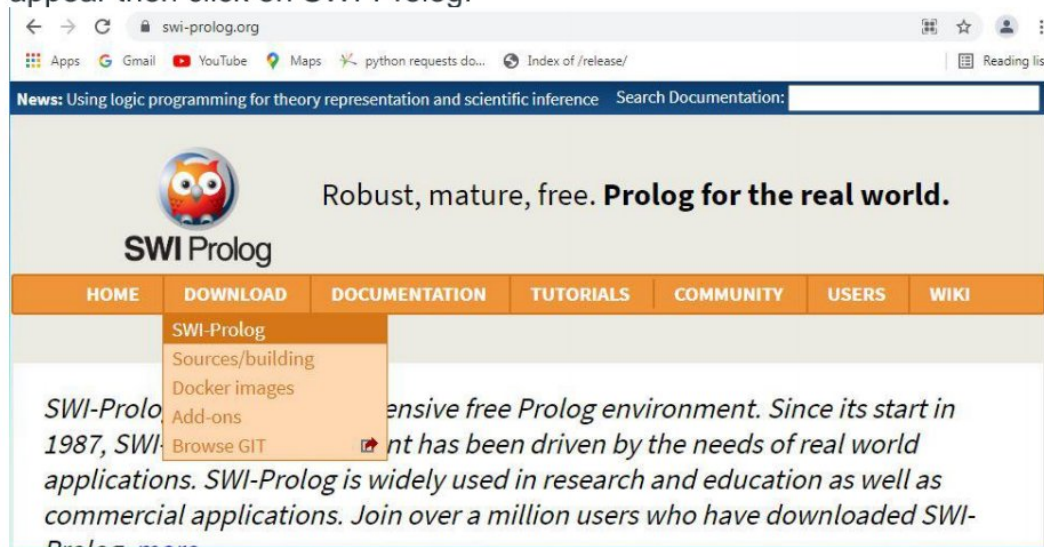
By querying the expert system, the system can infer the diagnosis by combining rules and facts, similar to how a human expert would deduce the best course of action.

Steps for Installing Prolog

Step 1: Visit swi-prolog.org website using any web browser.



Step 2: Click on Download which is adjacent to Home, dropdown list will appear then click on SWI-Prolog.



Step 3: New webpage will open, click on Stable release.



SWI-Prolog downloads

[HOME](#)
[DOWNLOAD](#)
[DOCUMENTATION](#)
[TUTORIALS](#)
[COMMUNITY](#)
[USERS](#)
[WIKI](#)

Available versions

The **stable** release is infrequently updated. It is fine for running basic Prolog code without surprises. The **development** version is released roughly every two to four weeks. This is the recommended version for developers and users of applications such as [SWISH](#) or [ClioPatria](#). Finally, the **GIT** and **daily** versions are for developers that want to contribute or have immediate access to patches. These versions are generally fine, but occasionally suffer from regression.

- [Stable release](#)
- [Development release](#)
- [Daily builds for Windows](#)
- Browse [GIT repository](#)

Read more about

- Available SWI-Prolog [versions](#)
- Information on [Linux packages and building on Linux](#)

Step 4: After clicking on stable release new webpage will open which will contain stable versions of prolog for different platforms. Under binaries there are two stable releases for windows, first is SWI-Prolog 8.4.0-1 for Microsoft Windows (64 bit) and the other is SWI-Prolog 8.4.0-1 for Microsoft Windows (32 bit). Click on the one as per your system configuration. Lets take the one for 64-bit operating system.

News: [logic programming for theory representation and scientific inference](#) Search Documentation:



Download SWI-Prolog stable versions

[HOME](#)
[DOWNLOAD](#)
[DOCUMENTATION](#)
[TUTORIALS](#)
[COMMUNITY](#)
[USERS](#)
[WIKI](#)

Linux versions are often available as a package for your distribution. We collect information about available packages and issues for building on specific distros [here](#). We provide a [PPA](#) for [Ubuntu](#) and [snap images](#)

Android binaries are available for [Termux](#) as the package `swi-prolog`. See also [Building SWI-Prolog on Android using LinuxOnAndroid](#)

Please check the [windows release notes](#) (also in the SWI-Prolog startup menu of your installed version) for details.

Examine the [Changelog](#).

Binaries		
	12,469,294 bytes	SWI-Prolog 8.4.0-1 for Microsoft Windows (64 bit) Self-installing executable for Microsoft's Windows 64-bit editions. Requires at least Windows 7. See the reference manual for deciding on whether to use the 32- or 64-bits version. This binary is linked against GMP 6.1.1 which is covered by the LGPL license. SHA256: 707f8de518dba327a6776e755470e49ee39917c443b4707b30f7fe44e6b5cecb
	12,452,305 bytes	SWI-Prolog 8.4.0-1 for Microsoft Windows (32 bit) Self-installing executable for MS-Windows. Requires at least Windows 7. Installs <code>swipl-win.exe</code> and <code>swipl.exe</code> . This binary is linked against GMP 6.1.1 which is covered by the LGPL license.

Step 5. After clicking on SWI-Prolog 8.4.0-1 for Microsoft Windows (64 bit), a new webpage will open, check on I understand checkbox to make the download link active. Then click on the download link, downloading of the executable file will start shortly. It is a small 11.9 MB file that will hardly take a minute.

News: [Using logic programming for theory representation and scientific inference](#) Search Documentation:

 **Download binary**

HOME DOWNLOAD DOCUMENTATION TUTORIALS COMMUNITY USERS WIKI

 Windows antivirus software works using *signatures* and *heuristics*. Using the huge amount of viruses and malware known today, arbitrary executables are often falsely classified as malicious. [Google Safe Browsing](#), used by most modern browsers, therefore often classifies our Windows binaries as malware. You can use e.g., [VirusTotal](#) to verify files with a large number of antivirus programs.

Our Windows binaries are cross-compiled on an isolated Linux container. The integrity of the binaries on the server is regularly verified by validating its SHA256 fingerprint.

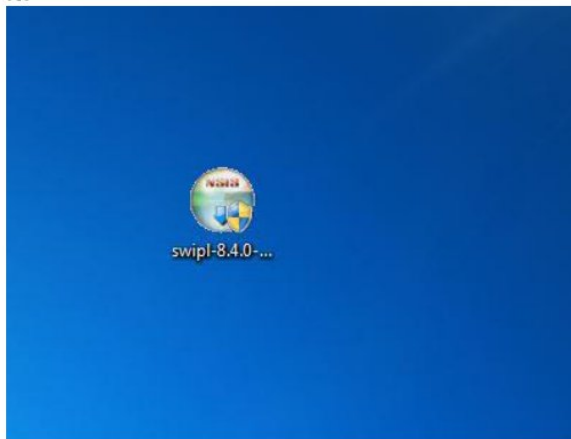
Please select the checkbox below to enable the actual download link.

☒ I understand

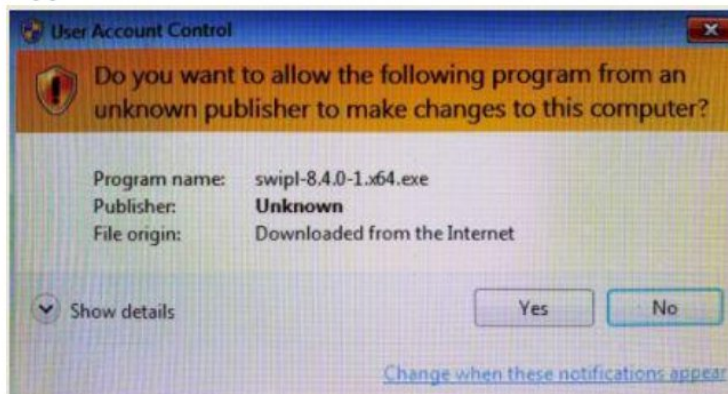
[Download swipl-8.4.0-1.x64.exe](#) (SHA256: 707f0de518dba327a6776e755470e49ee39917c443b4707b30f7fe44e6b5cceb)

[VIRUSTOTAL Scan Result](#)

Step 6: Now check for the executable file in downloads in your system and run it.



Step 7: It will prompt confirmation to make changes to your system. Click on Yes.



Step 8: Setup screen will appear, click on Next.



Step 9: The next screen will be of License Agreement, click on I Agree.



Step 10: After it there will be screen of installing options so check the box for Add swipl to the system path for all users, and also check the box for create a desktop icon and then click on the Next button.



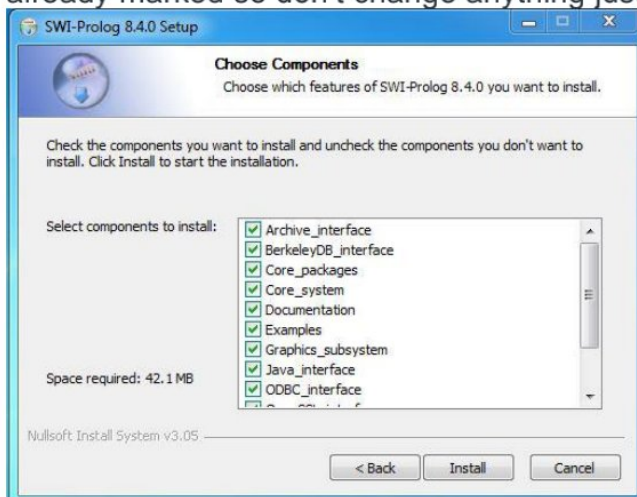
Step 11: The next screen will be of installing location so choose the drive which will have sufficient memory space for installation. It needed only a memory space of 50 MB.



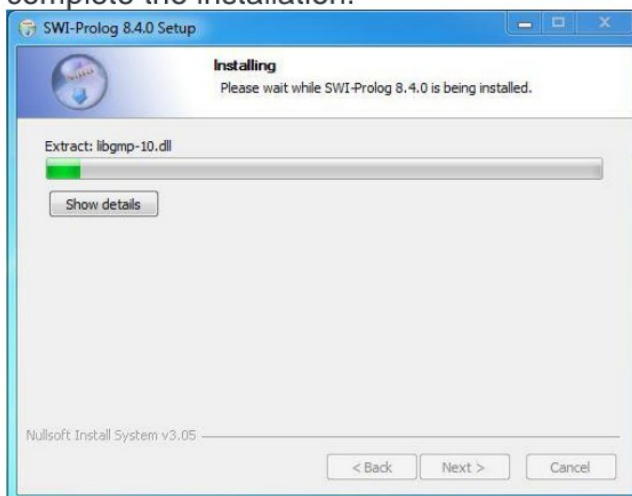
Step 12: Next screen will be of choosing Start menu folder so don't do anything just click on Next Button.



Step 13: This last screen is of choosing components, all components are already marked so don't change anything just click on Install button.



Step 14: After this installation process will start and will hardly take a minute to complete the installation.



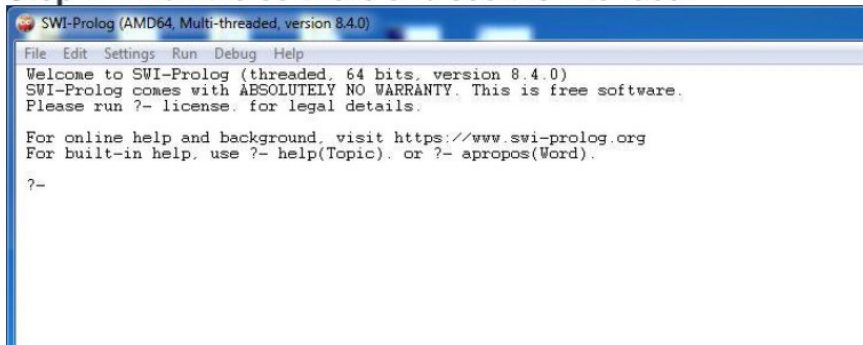
Step 15: Click on Finish after the installation process is complete.



Step 16: SWI Prolog is successfully installed on the system and an icon is created on the desktop.



Step 17: Run the software and see the interface.



Congratulations!! At this point, you have successfully installed SWI-Prolog on your windows system.

Hello World Program in Prolog

1. Save the following Prolog code in a file named `hello.pl`:

```
prolog
hello :-
write('Hello World!'), nl.
```

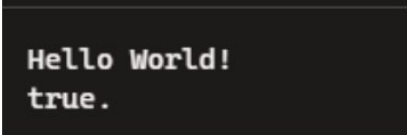
2. Load the program in your Prolog interpreter with:

```
prolog
?- consult('hello.pl').
```

3. Run the predicate `hello`:

```
prolog
?- hello.
```

This should print:



```
Hello World!
true.
```

The `nl` predicate is used to move to a new line after printing.

This demonstrates Prolog's ability to interact with the outside world by printing to the console.

VIVA QUESTIONS (EXP-1)

1. What are facts, rules, and queries in PROLOG?

- **Facts** represent known information. Example: `parent(john, mary).`
- **Rules** define relationships between facts. Example: `grandparent(X, Y) :- parent(X, Z), parent(Z, Y).`
- **Queries** are used to check information. Example: `?- grandparent(john, mary).`

2. What are predicates and clauses in PROLOG?

- A **predicate** defines a relation between objects. Example: `likes(john, pizza).`
- A **clause** is a single unit in PROLOG, either a fact, a rule, or a query.

3. What is an expert system in PROLOG?

- An **expert system** is a program that mimics human decision-making using a knowledge base and inference rules. PROLOG is widely used for expert systems due to its logical programming nature.

4. What are some key features of PROLOG?

- **Declarative language** (focuses on what to solve, not how).
- **Symbolic and Logical Processing** for AI applications.
- **Backtracking** to find multiple solutions.
- **Built-in pattern matching and recursion.**

5. How do you install and run a PROLOG program?

- Install **SWI-PROLOG** from www.swi-prolog.org.
- Write a PROLOG program (`hello.pl`) with `write('Hello, World!').`
- Run using `swipl -f hello.pl`.

6. What is Meta-Programming in PROLOG?

- Meta-programming in PROLOG refers to writing programs that manipulate or reason about other programs. It allows the use of **code as data**, enabling advanced AI applications like self-modifying code and automated reasoning.

7. What are the main symbols used in PROLOG, and what do they represent?

- `.` (Period): Ends a statement.
- `:-` (Colon-Dash): Defines a rule (if-then relationship).
- `,` (Comma): Represents logical AND.
- `;` (Semicolon): Represents logical OR.

EXPERIMENT-2

AIM: Write simple fact for the statements using PROLOG

- 1) Ram likes mango.
- 2) Seema is a girl.
- 3) Bill likes Cindy.
- 4) Rose is red.
- 5) John owns gold.

CODE:

```
% Facts
likes (Yash, mango).
girl(seema).
likes (bill, cindy).
red (rose).
owns (john,gold).
```


OUTPUT:

```
[mait@fedora Yash]$ swipl Exp2.pl

Warning: /home/mait/Documents/Yash/Exp2.pl:4:
Warning: Clauses of likes/2 are not together in the source-file
Warning: Earlier definition at /home/mait/Documents/Yash/Exp2.pl:2
Warning: Current predicate: girl/1
Warning: Use :- disjoint likes/2. to suppress this message

Welcome to SWI-Prolog (threaded, 64 bits, version 9.0.4)

SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

-----

Querying `likes(Yash, mango)`

?- likes(Yash, What).
What = mango.

?- likes(Who, mango).
Who = Yash.

?- likes(Yash, mango).
true.

?- likes(seema, mango).
false.

-----

Querying `girl(seema)`

?- girl(Who).
Who = seema.

?- girl(seema).
true.

?- girl(Yash).
false.
```

Querying `likes(bill, cindy)`

```
?- likes(bill, What).  
What = cindy.
```

```
?- likes(Who, cindy).  
Who = bill.
```

```
?- likes(bill, cindy).  
true.
```

```
?- likes(Yash, cindy).  
false.
```

Querying `red(rose)`

```
?- red(What).  
What = rose.
```

```
?- red(Who).  
Who = rose.
```

```
?- red(rose).  
true.
```

```
?- red(mango).  
false.
```

Querying `owns(john, gold)`

```
?- owns(john, What).  
What = gold.
```

```
?- owns(Who, gold).  
Who = john.
```

```
?- owns(john, gold).  
true.
```

```
?- owns(bill, gold).  
false.
```

```
?- owns(Who, What).  
Who = john,  
What = gold.
```

```
[maite@fedora prolog]$ echo "Yash Pandey 12214802722"  
Yash Pandey 12214802722
```

VIVA QUESTIONS (EXP-2)

1. How do you define a simple fact in PROLOG?

- A fact in PROLOG represents a basic assertion about some entity or relationship. It follows the syntax:

`fact_name(argument1, argument2, ...).`

Example: `likes(ram, mango).`

2. How would you write the given statements as facts in PROLOG?

- The given statements can be written as:

`likes(ram, mango).`

`girl(seema).`

`likes(bill, cindy).`

`color(rose, red).`

`owns(john, gold).`

3. What happens if we enter a fact in PROLOG without a period (.) at the end?

- PROLOG will not recognize the input as a complete statement and will wait for further input, showing an **unexpected end-of-file** or **incomplete clause** error.

4. How can we query PROLOG to check if a fact is true?

- We can use a query with a question mark and - (?-), like:

`?- likes(ram, mango).`

If the fact exists in the knowledge base, PROLOG will return true. Otherwise, it will return false.

5. Can PROLOG infer new facts from existing ones?

- No, PROLOG does not infer new facts automatically unless explicitly defined using **rules**. Facts only store direct knowledge.

6. What happens if we query PROLOG for a fact that is not present in the knowledge base?

- PROLOG will return false, indicating that the fact is not known.

Example:

`?- likes(john, mango).`

Output: false.

EXPERIMENT-3

AIM: Write predicates, one converts centigrade temperatures to Fahrenheit, the other checks if a temperature is below freezing using PROLOG

CODE:

```
c_to_f(C,F):- F is C * 9/5 + 32.  
freezing(F):- F =< 32.
```

OUTPUT:

```
[mait@fedora Yash]$ swipl Exp3.pl
```

```
Welcome to SWI-Prolog (threaded, 64 bits, version 9.0.4)
```

```
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.  
Please run ?- license. for legal details.
```

```
For online help and background, visit https://www.swi-prolog.org  
For built-in help, use ?- help(Topic). or ?- apropos(Word).
```

```
?- freezing(30).  
true.
```

```
?- freezing(36).  
false.
```

```
?- freezing(20).  
true.
```

```
?- c_to_f(20,F).  
F = 68.
```

```
?- c_to_f(-25,F).  
F = -13.
```

```
[mait@fedora Yash]$ echo "Yash Pandey 12214802722"  
Yash Pandey 12214802722
```


VIVA QUESTIONS (EXP-3)

1. How do you define a predicate in PROLOG?

- A predicate in PROLOG represents a relation between objects and is used to define facts or rules. It follows the syntax:

`predicate_name(arguments).`

2. How do you convert a temperature from Centigrade to Fahrenheit in PROLOG?

- We define a rule using the formula:

`c_to_f(C, F) :- F is (C * 9 / 5) + 32.`

This takes a Celsius value C and calculates the Fahrenheit value F.

3. How do you check if a temperature is below freezing (0°C) in PROLOG?

- We can define a rule like:

`below_freezing(C) :- C < 0.`

This checks if C is below 0°C and returns true if it is.

4. How do you query PROLOG to convert 25°C to Fahrenheit?

- We use the query:

`?- c_to_f(25, F).`

PROLOG will return `F = 77.`

5. What will be the output of the query `?- below_freezing(-5).` in PROLOG?

- Since -5°C is below freezing, PROLOG will return true.

6. What does the `is` keyword do in PROLOG calculations?

- The `is` keyword is used for arithmetic operations in PROLOG. It evaluates the right-hand expression and assigns it to the left-hand variable.

Example:

`F is (C * 9 / 5) + 32.`

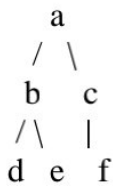
7. Can PROLOG automatically convert data types like strings and numbers in arithmetic operations?

- No, PROLOG does not perform implicit type conversions. If a value is not a number, arithmetic operations will result in an error.

Program - 4

Aim- Write a program to implement Breadth First Search Traversal.

Tree-



Code-

```
% Define the tree structure
```

```
tree(a, [b]).
```

```
tree(b, [c]).
```

```
tree(c, [d]).
```

```
tree(d, [e]).
```

```
tree(e, [f]).
```

```
tree(f, []).
```

```
% BFS traversal to find the path between two nodes
```

```
bfs(Start, Goal, Path) :-
```

```
    bfs_helper([[Start]], Goal, Path).
```

```
% Base case: If the goal is found in the first path, return it
```

```
bfs_helper([[Goal|Visited]]_, Goal, [Goal|Visited]).
```

```
% Recursive case: Explore the next level (children of the current node)
```

```
bfs_helper([Path|Paths], Goal, FinalPath) :-
```

```
    extend(Path, NewPaths),
```

```
    append(Paths, NewPaths, Paths1),
```

```
    bfs_helper(Paths1, Goal, FinalPath).
```

```
% Extend the current path by adding the neighbors of the current node (level-wise)
```

```
extend([Node|Path], NewPaths) :-
```

```
    findall([NewNode, Node|Path],
```

```
        (tree(Node, Neighbors), member(NewNode, Neighbors), \+  
member(NewNode, Path)),
```

```
        NewPaths).
```

```
% Predicate to read input from the user and run BFS level-wise traversal
run_bfs :-
    write('Enter start node: '),
    read(Start),
    write('Enter goal node: '),
    read(Goal),
    ( bfs(Start, Goal, Path)
    -> reverse(Path, ReversedPath), % Reverse to get the correct path order
        format('BFS Path from ~w to ~w: ~w~n', [Start, Goal, ReversedPath])
    ; format('No path found between ~w and ~w.~n', [Start, Goal])
    ).

% Example query to run the program
:- initialization(run_bfs).
```

Output-

```
?- [bfs_tree_input].
Enter start node: a
Enter goal node: f
BFS Path from a to f: [a, b, c, d, e, f]
```

```
?- [bfs_tree_input].
Enter start node: a
Enter goal node: e
BFS Path from a to e: [a, b, c, d, e]
```

```
?- [bfs_tree_input].
Enter start node: b
Enter goal node: f
No path found between b and f.
```

VIVA QUESTIONS (EXP-4)

1. What is Breadth-First Search (BFS) in Prolog?

Answer: BFS is a graph traversal algorithm that explores all neighbours of a node before moving to their children. In Prolog, it is implemented using recursion and list-based queue structures.

2. How does BFS differ from Depth-First Search (DFS)?

Answer: BFS explores nodes level by level using a queue (FIFO), ensuring the shortest path is found. DFS, on the other hand, explores as deep as possible using a stack (LIFO) before backtracking.

3. What are the key components of a BFS implementation in Prolog?

Answer: The key components include:

- A representation of the graph (edges as facts).
- A queue-based traversal mechanism using recursion.
- A visited list to avoid cycles.
- A base case to stop recursion when the goal is reached.

4. Why do we use lists to implement BFS in Prolog?

Answer: Since Prolog does not have built-in queues, lists are used to maintain the order of nodes to be explored, ensuring BFS traverses level by level.

5. How do you prevent infinite loops in BFS traversal?

Answer: By maintaining a **visited list**, which keeps track of nodes that have already been processed, preventing the algorithm from revisiting them.

6. What is the worst-case time complexity of BFS?

Answer: The worst-case time complexity of BFS is $O(V + E)$, where **V** is the number of vertices (nodes) and **E** is the number of edges in the graph.

7. Can BFS be used to find the shortest path in an unweighted graph?

Answer: Yes, BFS guarantees the shortest path in an **unweighted graph** because it explores all neighbours of a node before moving deeper, ensuring the first time a node is reached is via the shortest path.

EXPERIMENT-5

AIM: Write a Program to implement Water Jug Problem.

CODE

```
% jug capacities (X, Y)

% state is represented as (Amount in Jug1, Amount in Jug2)

% Initial state is (0, 0), and the goal is to reach (Z, _) or (_, Z)

solve_jug_problem(X, Y, Z) :-

    bfs([(0, 0)], X, Y, Z).

% BFS for exploring possible states

bfs([(State, Path)|_], X, Y, Z) :-

    goal(State, Z),

    print_solution(Path).

bfs([(State, Path)|Rest], X, Y, Z) :-

    next_state(State, X, Y, NextState),

    \+ member(NextState, Path), % avoid revisiting states

    bfs([NextState, (State, Path)], X, Y, Z).

% Goal check: If the goal is reached, print the path

goal((Z, _), Z).

goal(_, Z, Z).

% Generate the next valid state after applying an action

next_state((A, B), X, Y, (X, B)) :- % Fill Jug 1

    A < X.

next_state((A, B), X, Y, (A, Y)) :- % Fill Jug 2
```

$B < Y$.

next_state((A, B), X, Y, (0, B)) :- % Empty Jug 1

$A > 0$.

next_state((A, B), X, Y, (A, 0)) :- % Empty Jug 2

$B > 0$.

next_state((A, B), X, Y, (A1, B1)) :- % Pour Jug 1 into Jug 2

$A > 0$,

$B < Y$,

pour(A, B, X, Y, A1, B1).

next_state((A, B), X, Y, (A1, B1)) :- % Pour Jug 2 into Jug 1

$B > 0$,

$A < X$,

pour(B, A, Y, X, B1, A1).

pour(A, B, X, Y, A1, B1) :-

Transfer is $\min(A, Y - B)$,

A1 is $A - \text{Transfer}$,

B1 is $B + \text{Transfer}$.

print_solution(Path) :-

reverse(Path, RevPath),

print_steps(RevPath).

print_steps([]).

print_steps([(A, B)|Tail]) :-

write('('), write(A), write(','), write(B), write(')'), nl,

print_steps(Tail).

OUTPUT

```
[Yash@nixos:~/Repo/prolog]$ swipl jug_filling.pl
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.7)

SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.

Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- run.
Solutions Found:

Solution:
Step: [0,0]
Step: [4,0]
Step: [2,3]
Step: [2,0]
Step: [3,0]
Step: [3,3]
Step: [1,3]
```

```
State Possibilities for this Solution:
State [0,0]: Possible in this solution
State [4,0]: Possible in this solution
State [0,3]: Not possible in this solution
State [4,3]: Not possible in this solution
State [3,0]: Possible in this solution
State [1,3]: Possible in this solution
State [0,1]: Not possible in this solution
State [4,1]: Possible in this solution
State [2,0]: Possible in this solution
State [2,3]: Possible in this solution
State [3,3]: Possible in this solution
State [3,1]: Not possible in this solution
```

Solution:

Step: [0,0]

Step: [0,3]

Step: [3,0]

Step: [3,3]

Step: [4,2]

Step: [0,2]

Step: [2,0]

State Possibilities for this Solution:

State [0,0]: Possible in this solution

State [4,0]: Not possible in this solution

State [0,3]: Possible in this solution

State [4,3]: Not possible in this solution

State [3,0]: Possible in this solution

State [1,3]: Not possible in this solution

State [0,1]: Not possible in this solution

State [4,1]: Not possible in this solution

State [2,0]: Possible in this solution

State [2,3]: Not possible in this solution

State [3,3]: Possible in this solution

State [3,1]: Not possible in this solution

Solution:

Step: [0,0]

Step: [4,0]

Step: [1,3]

Step: [1,0]

Step: [0,1]

Step: [4,1]

Step: [2,3]

VIVA QUESTION (EXP-5)

1. What is the Water Jug Problem, and why is it important?

The Water Jug Problem is a puzzle where two jugs of different capacities are used to measure an exact amount of water. It is important in computer science as it demonstrates **state-space search, graph traversal algorithms (BFS, DFS), and constraint satisfaction problems**. It helps in problem-solving, AI, and optimization techniques.

2. How can the Water Jug Problem be solved?

The problem can be solved using:

1. **Brute Force** – Randomly try different actions.
2. **Graph-Based Search (BFS/DFS)** – Treat jug states as nodes and find the shortest path.
3. **Mathematical Approach** – Use GCD to check if a solution exists.
4. **Dynamic Programming** – Optimize state transitions to reduce redundant steps.

4. What are some real-world applications of the Water Jug Problem?

1. **Water distribution** – Optimizing resource allocation.
2. **Robotics** – Handling liquid measurement tasks.
3. **AI and game design** – Using state-space search for decision-making.
4. **Network load balancing** – Optimizing resource usage in cloud computing.
5. **Cryptography** – Using GCD-based logic in encryption algorithms.

5. What are some variations of the Water Jug Problem?

1. **Three-Jug Problem** – Three jugs instead of two.
2. **Limited Operations** – Restrictions like not emptying water.
3. **Finite Water Supply** – Constraints on available water.
4. **Time-Based Variations** – Solving the problem within a time limit.
5. **Multi-Agent Systems** – Multiple entities working together to solve the problem.

EXPERIMENT-6

AIM: Write a program to implement Tic-Tac-Toe Game.

//CODE

```
% Define the initial empty board
```

```
initial_board([empty, empty, empty, empty, empty, empty, empty, empty, empty])
```

```
% Define the game loop play :-
```

```
initial_board(Board), play_game(Board, x).
```

```
play_game(Board, Player) :- print_board(Board),
```

```
( winner(Board, x) -> % Check if X has won write('Player X wins!'), nl
```

```
; winner(Board, o) -> % Check if O has won write('Player O wins!'), nl
```

```
; full_board(Board) -> % Check if the board is full write('It\'s a draw!'), nl
```

```
; make_move(Board, Player, NewBoard), % Player makes a move next_player(Player,
```

```
NextPlayer), % Switch to the next player play_game(NewBoard, NextPlayer)
```

```
).
```

```
% Print the board
```

```
print_board([A, B, C, D, E, F, G, H, I]) :-
```

```
format('~w | ~w | ~w~n', [A, B, C]), write(' '), nl,
```

```
format('~w | ~w | ~w~n', [D, E, F]), write(' '), nl,
```

```
format('~w | ~w | ~w~n', [G, H, I])
```

% Check if a player has won winner(Board, Player) :-

(row_win(Board, Player); col_win(Board, Player); diag_win(Board, Player)

).

% Check for a winning row

row_win([A, B, C, _, _, _, _, _], Player) :- A = Player, B = Player, C = Player.

row_win([_, _, _, D, E, F, _, _, _], Player) :- D = Player, E = Player, F = Player.

row_win([_, _, _, _, _, _, G, H, I], Player) :- G = Player, H = Player, I = Player.

% Check for a winning column

col_win([A, _, _, D, _, _, G, _, _], Player) :- A = Player, D = Player, G = Player.

col_win([_, B, _, _, E, _, _, H, _], Player) :- B = Player, E = Player, H = Player.

col_win([_, _, C, _, _, F, _, _, I], Player) :- C = Player, F = Player, I = Player.

% Check for a winning diagonal diag_win([A, _, _, B, _, _, C], Player) :-

A = Player, B = Player, C = Player. diag_win([_, _, A, _, B, _, C, _, _], Player) :-

A = Player, B = Player, C = Player.

% Check if the board is full

full_board(Board) :-

\+ member(empty, Board).

% Make a move for the current player make_move(Board, Player, NewBoard) :-

write('Enter a position (1-9): '), read(Pos),

valid_move(Board, Pos),

replace(Pos, Player, Board, NewBoard).

% Validate the move (position must be between 1 and 9 and the cell must be empty)

valid_move(Board, Pos) :-

Pos >= 1, Pos <= 9,

nth1(Pos, Board, X), X = empty.

% Replace a position on the board with the current players symbol replace(Pos, Player,
Board, NewBoard) :-

nth1(Pos, Board, _, TempBoard), nth1(Pos, NewBoard, Player, TempBoard).

% Switch to the next player next_player(x, o). next_player(o, x).

OUTPUT

```
[~@nixos:~/Repo/prolog]$ swipl ticTacToe.pl
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.7)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- play.
empty | empty | empty
-----
empty | empty | empty
-----
empty | empty | empty
Enter a position (1-9): 1
|:
x | empty | empty
-----
empty | empty | empty
-----
empty | empty | empty
|: 2.

x | o | empty
-----
empty | empty | empty
Enter a position (1-9): |: 5.
x | o | empty
-----
empty | x | empty
-----
empty | empty | empty
|: 8.
x | o | empty
-----
empty | x | empty
-----
empty | o | empty
Enter a position (1-9): |: 9.
x | o | empty
-----
empty | x | empty
-----
empty | o | x
Player X wins!
true.
```

```

?- play.
empty | empty | empty
-----
empty | empty | empty
-----
empty | empty | empty
1.

x | empty | empty
-----
empty | empty | empty
-----
empty | empty | empty
Enter a position (1-9): |: 3.

x | empty | o
-----
empty | empty | empty
-----
empty | empty | empty
Enter a position (1-9): |: 4.

x | empty | o
-----
x | empty | empty
-----
empty | empty | empty
Enter a position (1-9): |: 6.

```

```

x | empty | o
-----
empty | empty | empty
Enter a position (1-9): |: 2.

x | x | o
-----
x | empty | o
-----
empty | empty | empty
Enter a position (1-9): |: 9.

x | x | o
-----
x | empty | o
-----
empty | empty | o
Player 0 wins!

true.

```



```
$ consult("/Users/priyanshu/Desktop/h1.pl").
```

```
true.
```

```
?- initial_board(Board), play_game([[1,1],[2,1],[1,2],[2,2],[1,3]], Board, 'X', FinalBoard).
```

```
After move: X at (1,1):
```

```
X - -  
- - -  
- - -
```

```
After move: O at (2,1):
```

```
X - -  
O - -  
- - -
```

```
After move: X at (1,2):
```

```
X X -  
O - -  
- - -
```

```
After move: O at (2,2):
```

```
X X -  
O O -  
- - -
```

```
After move: X at (1,3):
```

```
X X X  
O O -  
- - -
```

```
Final Board:
```

```
X X X  
O O -  
- - -
```

```
Result: X
```

```
Board = [['X', 'X', 'X'], ['O', 'O', '-'], ['- ', '- ', '- ']]
```

```
FinalBoard = [['X', 'X', 'X'], ['O', 'O', '-'], ['- ', '- ', '- ']]
```

```
Result = 'X'.
```

```
?- initial_board(Board), play_game([[2,1],[1,1],[2,2],[1,2],[2,3]], Board, 'X', FinalBoard).
```

```
After move: X at (2,1):
```

```
- - -  
X - -  
- - -
```

```
After move: O at (1,1):
```

```
O - -  
X - -  
- - -
```

```
After move: X at (2,2):
```

```
O - -  
X X -  
- - -
```

```
After move: O at (1,2):
```

```
O O -  
X X -  
- - -
```

```
After move: X at (2,3):
```

```
O O -  
X X X  
- - -
```

```
| [system:~/Repo/prolog]$ swipl ticTacToe.pl
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.7)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.
```

```
For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).
```

```
?- play.
```

```
empty | empty | empty
```

```
-----
```

```
empty | empty | empty
```

```
-----
```

```
empty | empty | empty
```

```
Enter a position (1-9): 1.
```

```
x | empty | empty
```

```
-----
```

```
empty | empty | empty
```

```
-----
```

```
empty | empty | empty
```

```
Enter a position (1-9): 3.
```

```
x | empty | o
```

```
-----
```

```
empty | empty | empty
```

```
-----
```

```
empty | empty | empty
```

```
Enter a position (1-9): 2.
```

```
x | x | o
```

```
-----
```

```
empty | empty | empty
```

```
-----
```

```
empty | empty | empty
```

```
Enter a position (1-9): 4.
```

```
x | x | o
```

```
-----
```

```
o | empty | empty
```

```
-----
```

```
empty | empty | empty
```

```
Enter a position (1-9): 6.
```

```
| [nixos:~/Repo/prolog]$ swipl ticTacToe.pl
```

```
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.7)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.
```

```
For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).
```

```
?- play.
```

```
empty | empty | empty
```

```
-----
```

```
empty | empty | empty
```

```
-----
```

```
empty | empty | empty
```

```
Enter a position (1-9): 10.
```

```
false.
```

```
x | x | o
```

```
-----
```

```
o | empty | x
```

```
-----
```

```
empty | empty | empty
```

```
Enter a position (1-9): 5.
```

```
x | x | o
```

```
-----
```

```
o | o | x
```

```
-----
```

```
empty | empty | empty
```

```
Enter a position (1-9): 7.
```

```
x | x | o
```

```
-----
```

```
o | o | x
```

```
-----
```

```
x | empty | empty
```

```
Enter a position (1-9): 9.
```

```
x | x | o
```

```
-----
```

```
o | o | x
```

```
-----
```

```
x | empty | o
```

```
Enter a position (1-9): 8.
```

```
x | x | o
```

```
-----
```

```
o | o | x
```

```
-----
```

```
x | o | x
```

```
It's a draw!
```

```
false.
```

VIVA QUESTIONS (EXP-6)

1. What is the significance of AI in the Tic-Tac-Toe game?

AI in Tic-Tac-Toe is used to make intelligent decisions by evaluating possible moves and selecting the best one. It ensures optimal gameplay using algorithms like Minimax, which simulates all possible moves and counter-moves. The AI can also incorporate heuristics to make efficient decisions. This helps in developing an unbeatable or competitive opponent.

2. How does the Minimax algorithm work in Tic-Tac-Toe?

The Minimax algorithm is a decision-making strategy that explores all possible game states. It assigns scores to terminal states: +10 for a win, -10 for a loss, and 0 for a draw. The AI assumes that the opponent plays optimally, minimizing AI's maximum possible loss. The algorithm recursively evaluates moves and selects the one leading to the best outcome.

3. What are the different ways to represent the Tic-Tac-Toe board in a program?

The board can be represented using a 2D array, lists, or a dictionary. A 3×3 matrix is the most common approach, where each cell stores 'X', 'O', or empty values. Some implementations use a single-dimensional array indexed from 1-9. Another approach is a graph-based representation for more complex AI logic.

4. How do you check for a winning condition in Tic-Tac-Toe?

The program checks if any row, column, or diagonal has the same symbol ('X' or 'O'). This can be done using loops or predefined sets of indices representing winning combinations. If any of these conditions are met, the game declares a winner; otherwise, it continues or results in a draw.

5. What improvements can be made to a basic Tic-Tac-Toe AI?

The AI can be enhanced using Alpha-Beta Pruning to optimize Minimax, reducing the number of nodes evaluated. Machine learning can be integrated to adapt AI strategies based on past games. Another improvement is incorporating randomness to make the AI less predictable. Enhancing the user interface also improves the overall experience.

EXPERIMENT-7

AIM: Write a program to remove punctuations from the given string.

//CODE

```
% Remove punctuation from a list of character codes
remove_punct([], []).
remove_punct([Char|Rest], Filtered) :-
    ( (char_type(Char, alnum) ; char_type(Char, space)) ->
      Filtered = [Char|FilteredRest]
    ; Filtered = FilteredRest
    ),

    remove_punct(Rest, FilteredRest).

% Main predicate to run the program

main :-

    write('Enter a string: '),
    read_line_to_codes(user_input, InputCodes),
    remove_punct(InputCodes, FilteredCodes),
    string_codes(FilteredString, FilteredCodes),
    write('Filtered string: '), write(FilteredString), nl.
```

OUTPUT:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.9)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.
```

```
For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).
```

```
?- consult("/Users/yashpandey/Desktop/hi.pl").
true.
```

```
?- main.
Enter a string: !!!!!??!
Filtered string:
```

```
true.
```

```
?- main. |
Enter a string: Yash,Pandey!@9!?7AI!ML
Filtered string: YashPandey097AIML
```

```
true.
```

```
?- main.
Enter a string: ""
Filtered string:
```

```
true.
```

```
?- main.
Enter a string:
Filtered string:
```

```
true.
```

```
?-
```


VIVA QUESTIONS(EXP-7)

1. What is the objective of your experiment?

The experiment aims to develop a Prolog program that removes punctuation marks from a string. It ensures that only alphanumeric characters are retained. This preprocessing is useful for tasks like text analysis in AI. By cleaning the text, we improve its quality for further processing. It's an essential step for many natural language processing (NLP) tasks.

2. Why did you choose Prolog for this task?

Prolog is well-suited for pattern matching and symbolic manipulation. Its list processing abilities make it efficient for text filtering tasks. Using Prolog simplifies the rule-based removal of unwanted characters. Prolog's recursion capabilities help in effectively scanning and modifying strings. It's commonly used in AI, which makes it appropriate for this experiment.

3. How does your Prolog program work?

The program reads a string and filters out non-alphanumeric characters. It recursively checks each character, retaining letters and digits while removing punctuation. Prolog's list processing helps manage this efficiently. The final output is a clean string, free from unwanted symbols. This enhances the structure and readability of the text.

4. What challenges did you face in implementing this?

A key challenge was working with Prolog's list-based string processing. Ensuring only punctuation marks are removed without affecting meaningful symbols was tricky. Recursive filtering required attention to avoid performance issues. Debugging Prolog's output can be complex, especially for beginners. However, iterative improvements resolved these challenges.

5. What are the real-world applications of this experiment?

This technique is useful in AI fields like Natural Language Processing (NLP). It helps clean text for tasks like sentiment analysis, spam detection, and chatbot development. Clean data improves model performance in machine learning applications. It's also valuable in cybersecurity and data analysis. Text preprocessing is vital for many AI systems.

EXPERIMENT-8

AIM: Write a prolog program to sort a sentence.

//CODE

% Split a sentence into words, sort them, and display the sorted sentence

:- initialization(main).

% Convert a sentence into a list of words

split_sentence(Sentence, WordList) :-

atomic_list_concat(Words, ' ', Sentence),

maplist(atom_string, Words, WordList).

% Sort the list of words

sort_sentence(Sentence, SortedSentence) :-

split_sentence(Sentence, WordList),

sort(WordList, SortedWords),

atomic_list_concat(SortedWords, ' ', SortedSentence).

% Main function to take input and display output

main :-

write('Enter a sentence: '),

read_line_to_string(user_input, Sentence),

sort_sentence(Sentence, SortedSentence),

write('Sorted sentence: '), write(SortedSentence), nl.

OUTPUT:

```
[yash@nixos:~/Repo/prolog]$ swipl sort.pl
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.7)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- sort_sentence("Hello World! This is Yash!", Sort).
Sort = "Hello Yash! This World! is".
```

```
[yash@nixos:~/Repo/prolog]$ swipl sort.pl
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.7)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- sort_sentence("Helo world. Today is our Prolog lab.", Result).
Result = "Helo Prolog Today is lab. our world.".
```

VIVA QUESTION (EXP-8)

1. What is the purpose of a Prolog program to sort a sentence?

The purpose of this program is to take a sentence as input, break it into words, and arrange them in sorted order. Prolog uses built-in predicates like `split_string/4` and `msort/2` for this task. The sorted sentence is returned as output while maintaining punctuation if needed. This helps in understanding Prolog's list processing capabilities.

2. How does Prolog handle string manipulation in the sorting process?

Prolog treats strings as lists of characters or words, making them easy to manipulate. The `split_string/4` predicate is used to break a sentence into words. The words are stored as a list, which is then sorted using `msort/2`. Finally, the sorted words are concatenated back into a string using `atomic_list_concat/3`.

3. What is the role of the `msort/2` predicate in sorting a sentence?

The `msort/2` predicate in Prolog is used to sort a list of elements in ascending order. It removes duplicate elements if they exist in the input list. In a sentence sorting program, it arranges words alphabetically while ignoring punctuation marks if not included in sorting. This ensures proper ordering of words within the sentence.

4. Can Prolog sort words while maintaining their original case?

By default, Prolog sorts words in a case-sensitive manner, meaning uppercase letters come before lowercase ones. To achieve case-insensitive sorting, words can be converted to lowercase before sorting. After sorting, the original cases can be restored using additional mapping functions. This ensures meaningful sorting in natural language processing.

5. How can this Prolog sorting program be modified to handle punctuation?

To handle punctuation, special characters should be separated from words before sorting. This can be done using regex-based splitting or filtering specific characters. After sorting, punctuation marks can be reattached to their respective words. This ensures that the meaning and readability of the sentence remain intact.

EXPERIMENT-9

AIM: Write a program to implement Hangman Game in Prolog.

//CODE

```
% Main predicate to start the Hangman game
hangman :-
write('Welcome to Hangman!'), nl,
write('Player 1, enter the secret word followed
by a period (e.g., hello.): '),
read(SecretAtom),
downcase_atom(SecretAtom, LowerSecret),
atom_chars(LowerSecret, Word),
% Print instructions for Player 2
print_instructions,
length(Word, Len),
write('Player 2, the word has '), write(Len),
write(' letters. '), nl,
write('You have 6 chances to guess the word. '),
nl,
write('Guess one letter at a time. '), nl,
% Initialize the state with underscores
length(State, Len),
maplist(=('_'), State),
Chances = 6,
Guessed = [],
% Start the game loop
game_loop(State, Word, Chances, Guessed).
% Predicate to print instructions for Player 2
print_instructions :-
write('Instructions for Player 2:'), nl,
write('1. The goal is to guess the secret word
chosen by Player 1. '), nl,
write('2. You will guess one letter at a time. '),
```



```

nl,
write('3. For each correct guess, the letter will
be revealed in its correct position(s).'), nl,
write('4. For each incorrect guess, you will lose
one chance. '), nl,
write('5. You have a total of 6 chances to guess
the word. '), nl,
write('6. If you guess the word correctly before
running out of chances, you win. '), nl,
write('7. If you run out of chances, you lose,
and the secret word will be revealed. '), nl,
write('8. After each guess, a separator line will
be displayed to distinguish between guesses. '), nl,
write('Good luck!'), nl.

% Game loop handles the recursive logic of the game
game_loop(State, Word, Chances, Guessed) :-
( fully_guessed(State) ->
atomic_list_concat(State, ", WordAtom),
format('Congratulations! You guessed the
word: ~w~n', [WordAtom])
; Chances = 0 ->
atomic_list_concat(Word, ", WordAtom),
format('Game over! You ran out of chances.
The word was: ~w~n', [WordAtom])
; display_state(State, Chances, Guessed),
get_guess(Guess),
( member(Guess, Guessed) ->
write('You already guessed that
letter. '), nl,
write('-----'), nl,
game_loop(State, Word, Chances, Guessed)
; member(Guess, Word) ->
update_state(Word, State, Guess,
NewState),

```

```

write('Correct guess!'), nl,
write('-----'), nl,
game_loop(NewState, Word, Chances, [Guess
| Guessed])
; NewChances is Chances - 1,
write('Wrong guess!'), nl,
write('-----'), nl,
game_loop(State, Word, NewChances, [Guess
| Guessed])
)).

% Check if the word is fully guessed (no underscores
remain)
fully_guessed(State) :-
\+ member('_', State).

% Display the current game state
display_state(State, Chances, Guessed) :-
atomic_list_concat(State, ' ', StateAtom),
format('Current word: ~w~n', [StateAtom]),
format('Chances left: ~w~n', [Chances]),
format('Guessed letters: ~w~n', [Guessed]).

% Get a guess from the user
get_guess(Guess) :-
write('Guess a letter: '),
read(GuessAtom),
downcase_atom(GuessAtom, LowerGuess),
atom_chars(LowerGuess, [Guess | _]).

% Update the state by revealing the guessed letter in
correct positions
update_state(Word, OldState, Letter, NewState) :-
maplist(update_letter(Letter), Word, OldState,
NewState).

% Helper predicate to update a single position in the
state
update_letter(Letter, ActualLetter, OldStateLetter,

```

```
NewStateLetter) :-  
( ActualLetter = Letter ->  
NewStateLetter = Letter  
; NewStateLetter = OldStateLetter  
).
```

OUTPUT

```
?- hangman.  
Welcome to Hangman!  
Player 1, enter the secret word followed by a period (e.g., hello.): mango.  
Instructions for Player 2:  
1. The goal is to guess the secret word chosen by Player 1.  
2. You will guess one letter at a time.  
3. For each correct guess, the letter will be revealed in its correct position(s).  
4. For each incorrect guess, you will lose one chance.  
5. You have a total of 6 chances to guess the word.  
6. If you guess the word correctly before running out of chances, you win.  
7. If you run out of chances, you lose, and the secret word will be revealed.  
8. After each guess, a separator line will be displayed to distinguish between guesses.  
Good luck!  
Player 2, the word has 5 letters.  
You have 6 chances to guess the word.  
Guess one letter at a time.  
Current word: _ _ _ _ _  
Chances left: 6  
Guessed letters: []  
Guess a letter: |: m.  
Correct guess!
```

```
-----  
Current word: m _ _ _ _  
Chances left: 6  
Guessed letters: [m]  
Guess a letter: |: a.  
Correct guess!  
-----  
Current word: m a _ _ _  
Chances left: 6  
Guessed letters: [a,m]  
Guess a letter: |: x.  
Wrong guess!  
-----  
Current word: m a _ _ _  
Chances left: 5  
Guessed letters: [x,a,m]  
Guess a letter: |: a.  
You already guessed that letter.
```

```
Current word: m a _ _ _  
Chances left: 5  
Guessed letters: [x,a,m]  
Guess a letter: |: n.  
Correct guess!  
-----  
Current word: m a n _ _  
Chances left: 5  
Guessed letters: [n,x,a,m]  
Guess a letter: |: g.  
Correct guess!  
-----  
Current word: m a n g _  
Chances left: 5  
Guessed letters: [g,n,x,a,m]  
Guess a letter: |: o.  
Correct guess!  
-----  
Congratulations! You guessed the word: mango  
true.
```

?- hangman.

Welcome to Hangman!

Player 1, enter the secret word followed by a period (e.g., hello.): marvel.

Instructions for Player 2:

1. The goal is to guess the secret word chosen by Player 1.
 2. You will guess one letter at a time.
 3. For each correct guess, the letter will be revealed in its correct position(s).
 4. For each incorrect guess, you will lose one chance.
 5. You have a total of 6 chances to guess the word.
 6. If you guess the word correctly before running out of chances, you win.
 7. If you run out of chances, you lose, and the secret word will be revealed.
 8. After each guess, a separator line will be displayed to distinguish between guesses.
- Good luck!

Player 2, the word has 6 letters.

You have 6 chances to guess the word.

Guess one letter at a time.

Current word: _ _ _ _ _

Chances left: 6

Guessed letters: []

Guess a letter: |: e.

Correct guess!

Current word: _ _ _ _ e _

Chances left: 6

Guessed letters: [e]

Guess a letter: |: l.

Correct guess!

Current word: _ _ _ _ e l

Chances left: 6

Guessed letters: [l,e]

Guess a letter: |: s.

Wrong guess!

Current word: _ _ _ _ e l

Chances left: 5

Guessed letters: [s,l,e]

Guess a letter: |: g.

Wrong guess!

Current word: _ _ _ _ e l

Chances left: 4

Guessed letters: [g,s,l,e]

Guess a letter: |: p.

Wrong guess!

```
-----
Current word: _ _ _ _ e l
Chances left: 3
Guessed letters: [p,g,s,l,e]
Guess a letter: |: l.
You already guessed that letter.
-----
Current word: _ _ _ _ e l
Chances left: 3
Guessed letters: [p,g,s,l,e]
Guess a letter: |: f.
Wrong guess!
-----
Current word: _ _ _ _ e l
Chances left: 2
Guessed letters: [f,p,g,s,l,e]
Guess a letter: |: u.
Wrong guess!
-----
Current word: _ _ _ _ e l
Chances left: 1
Guessed letters: [u,f,p,g,s,l,e]
Guess a letter: |: h.
Wrong guess!
-----
Game over! You ran out of chances. The word was: marvel
true.
```

VIVA QUESTIONS(EXP-9)

- **Q: What is the main logic behind implementing Hangman in Prolog?**

A: Hangman in Prolog works by recursively processing user guesses, updating the guessed word, and checking if the word has been fully guessed or if the player has exhausted all attempts. It uses facts and rules to store the secret word, guessed letters, and remaining chances, updating them based on correct or incorrect guesses.

- **Q: How does Prolog handle user input and update the game state dynamically?**

A: Prolog reads user input using predicates like `read/1` and updates the game state using recursive rules. Each turn, it checks if the guessed letter is in the word, updates the displayed word accordingly, and either continues the game or ends it based on the remaining attempts.

- **Q: What Prolog data structures are used to store the game state?**

A: Lists are used to store the secret word, guessed letters, and the current state of the word (e.g., `_ _ _ e _ l`). Counters for remaining attempts are maintained as numeric values that decrement upon incorrect guesses. Facts and recursion help modify these dynamically.

- **Q: How does Prolog determine if the game is won or lost?**

A: The game is won when all letters in the secret word have been correctly guessed, which is checked by comparing the guessed word list with the original word. The game is lost when the remaining attempts reach zero, at which point the program displays the correct word and ends execution.

- **Q: Why is Prolog a suitable choice for implementing Hangman?**

A: Prolog is suitable because it efficiently handles pattern matching and recursion, which are key to checking guessed letters, updating the word state, and controlling game flow. Its logical inference mechanism simplifies decision-making, making it a good choice for text-based games like Hangman.

EXPERIMENT-10

AIM: Write a program to implement hangman using python.

//CODE

```
import os

def hangman():

    print("Welcome to Hangman!")

    secret_word = input("Player 1, enter the secret
word: ").lower()

    # Clear the screen (simulating hiding the word in
Prolog with instructions)

    os.system('cls' if os.name == 'nt' else 'clear')

    # Print instructions for Player 2

    print_instructions()

    # Initialize game state

    word = list(secret_word)

    state = ['_'] * len(word)

    chances = 6

    guessed = []

    print(f"Player 2, the word has {len(word)}
letters.")

    print("You have 6 chances to guess the word.")

    print("Guess one letter at a time.")

    # Start the game loop

    game_loop(state, word, chances, guessed)

def print_instructions():

    print("Instructions for Player 2:")

    print("1. The goal is to guess the secret word
chosen by Player 1.")

    print("2. You will guess one letter at a time.")

    print("3. For each correct guess, the letter will
be revealed in its correct position(s).")

    print("4. For each incorrect guess, you will lose
```

```
one chance.")

print("5. You have a total of 6 chances to guess
the word.")

print("6. If you guess the word correctly before
running out of chances, you win.")

print("7. If you run out of chances, you lose,
and the secret word will be revealed.")

print("8. After each guess, a separator line will
be displayed to distinguish between guesses.")

print("Good luck!")

def game_loop(state, word, chances, guessed):
    while True:
        # Check win or lose conditions
        if fully_guessed(state):
            word_str = ".join(state)
            print(f'Congratulations! You guessed the
            word: {word_str}')
            break
        elif chances == 0:
            word_str = ".join(word)
            print(f'Game over! You ran out of
            chances. The word was: {word_str}')
            break
        # Display current state
        display_state(state, chances, guessed)
        # Get guess from Player 2
        guess = get_guess()
        # Process the guess
        if guess in guessed:
            print("You already guessed that letter.")
            print("-----")
        elif guess in word:
            update_state(word, state, guess)
            print("Correct guess!")
```

```

guessed.append(guess)
print("-----")
else:
    chances -= 1
    print("Wrong guess!")
    guessed.append(guess)
    print("-----")
def fully_guessed(state):
    return '_' not in state
def display_state(state, chances, guessed):
    print("Current word:", " ".join(state))
    print(f"Chances left: {chances}")
    print(f"Guessed letters: {guessed}")
def get_guess():
    while True:
        guess = input("Guess a letter: ").lower()
        if len(guess) == 1 and guess.isalpha():
            return guess
        print("Invalid input! Please enter a single letter.")
def update_state(word, state, letter):
    for i in range(len(word)):
        if word[i] == letter:
            state[i] = letter
# Start the game
if __name__ == "__main__":
    hangman()

```


OUTPUT

Welcome to Hangman!

YashPandey, enter the secret word: mango

Instructions for Player 2:

1. The goal is to guess the secret word chosen by YashPandey.
2. You will guess one letter at a time.
3. For each correct guess, the letter will be revealed in its correct position(s).
4. For each incorrect guess, you will lose one chance.
5. You have a total of 6 chances to guess the word.
6. If you guess the word correctly before running out of chances, you win.
7. If you run out of chances, you lose, and the secret word will be revealed.
8. After each guess, a separator line will be displayed to distinguish between guesses.

Good Luck!

Player 2, the word has 5 letters.

You have 6 chances to guess the word.

Guess one letter at a time.

Current word: _ _ _ _ _

Chances left: 6

Guessed Letters: []

Guess a letter: n

Correct guess!

Current word: n _ _ _ _

Chances left: 6

Guessed Letters: ['n']

Guess a letter: a

Correct guess!

Current word: n a _ _ _

Chances left: 6

Guessed Letters: ['n', 'a']

Guess a letter: h

Wrong guess!

Current word: n a _ _ _

Chances left: 5

Guessed Letters: ['n', 'a', 'h']

Guess a letter: o

Correct guess!

Current word: n a _ o _

Chances left: 5

Guessed Letters: ['n', 'a', 'h', 'o']

Guess a letter: o

You already guessed that letter.

Current word: n a _ o _

Chances left: 5

Guessed Letters: ['n', 'a', 'h', 'o']

Guess a letter: g

Correct guess!

Current word: n a _ g o

Chances left: 5

Guessed Letters: ['n', 'a', 'h', 'o', 'g']

Guess a letter: m

Correct guess!

Congratulations! You guessed the word: mango

Welcome to Hangman!

YashPandey, enter the secret word: hangman

Instructions for Player 2:

1. The goal is to guess the secret word chosen by YashPandey.
2. You will guess one letter at a time.
3. For each correct guess, the letter will be revealed in its correct position(s).
4. For each incorrect guess, you will lose one chance.
5. You have a total of 6 chances to guess the word.
6. If you guess the word correctly before running out of chances, you win.
7. If you run out of chances, you lose, and the secret word will be revealed.
8. After each guess, a separator line will be displayed to distinguish between guesses.

Good Luck!

Player 2, the word has 7 letters.

You have 6 chances to guess the word.

Guess one letter at a time.

Current word: _ _ _ _ _

Chances left: 6

Guessed Letters: []

Guess a letter: s

Wrong guess!

Current word: _ _ _ _ _

Chances left: 5

Guessed Letters: ['s']

Guess a letter: t

Wrong guess!

Current word: _ _ _ _ _

Chances left: 4

Guessed Letters: ['s', 't']

Guess a letter: e

Wrong guess!

Current word: _ _ _ _ _

Chances left: 3

Guessed Letters: ['s', 't', 'e']

Guess a letter: o

Wrong guess!

Current word: _ _ _ _ _

Chances left: 2

Guessed Letters: ['s', 't', 'e', 'o']

Guess a letter: r

Wrong guess!

Current word: _ _ _ _ _

Chances left: 1

Guessed Letters: ['s', 't', 'e', 'o', 'r']

Guess a letter: l

Wrong guess!

Oh no! You've run out of chances.

The correct word was: **hangman**

Better luck next time!

VIVA QUESTION(EXP-10)

1. What is the basic logic behind the Hangman game in Python?

The Hangman game works by letting a player guess letters of a hidden word. For each correct guess, the letter is revealed in its position(s). For wrong guesses, the player loses one of the limited number of chances. The game continues until the player either guesses the full word correctly or runs out of attempts.

2. How do you handle user inputs and check if the guessed letter is correct?

User inputs are taken using the `input()` function. The guessed letter is checked using a simple condition: `if letter in word`. If it's present, we reveal it in the appropriate places in the word display; otherwise, we reduce the remaining attempts. We also store guessed letters to avoid repetition.

3. How do you display the progress of the guessed word during the game?

We use a list of underscores (e.g., `['_', '_', '_']`) representing the hidden word. As the player guesses correct letters, we update the list by replacing the underscores with the correct letters at their corresponding positions. The list is printed each round to show progress.

4. How do you keep track of the number of chances left for the player?

We initialize a variable, e.g., `chances = 6`, and decrement it each time the player makes an incorrect guess. The game ends when the variable reaches zero. We display the remaining chances after each guess to keep the player informed.

5. Can you explain how to detect a win or loss condition in the game?

A win is detected when the list of guessed letters matches the full word, meaning all letters have been guessed correctly. A loss is detected when the number of chances becomes zero before guessing the full word. Both conditions trigger end messages accordingly.

EXPERIMENT-11

AIM: Write a program to remove stop words for a given passage from a text file using NLTK.

//CODE

```
import nltk
import string

nltk.download('stopwords', quiet=True)
from nltk.corpus import stopwords

def remove_stop_words(input_file, output_file):
    stop_words = set(stopwords.words('english'))

    try:
        with open(input_file, 'r', encoding='utf-8') as file:
            text = file.read()

            text = text.translate(str.maketrans("", "", string.punctuation)).lower()

            words = text.split()

            filtered_words = [word for word in words if word not in stop_words]

            filtered_text = ' '.join(filtered_words)

            with open(output_file, 'w', encoding='utf-8') as file:
                file.write(filtered_text)

            print(f'Stop words removed. Result written to {output_file}')

    except FileNotFoundError:
        print(f'Error: Input file {input_file} not found.')
    except Exception as e:
```



```
print(f'An error occurred: {e}')
```

```
def main():
```

```
    input_file = './input.txt'
```

```
    output_file = './output.txt'
```

```
    remove_stop_words(input_file, output_file)
```

```
if __name__ == "__main__":
```

```
    main()
```

OUTPUT

 input.txt

```
1 Artificial intelligence is the simulation of human intelligence
  processes by machines, especially computer systems.
2 These processes include learning, reasoning, and self-correction.
3 AI is used in expert systems, natural language processing, and
  machine vision.
4
```

```
PS C:\Users\HP\Desktop\AI EXPERIMENTS> python exp11.py
```

```
Stop words removed. Result written to ./output.txt
```

```
PS C:\Users\HP\Desktop\AI EXPERIMENTS> 
```

output.txt

```
1 artificial intelligence simulation human intelligence processes
  machines especially computer systems processes include learning
  reasoning selfcorrection ai used expert systems natural language
  processing machine vision
```

- All stop words like is, the, by, and, in, of, etc. are removed.
- Text is converted to lowercase and punctuation is stripped.
- Words like self-correction become selfcorrection due to punctuation removal

VIVA QUESTIONS(EXP-11)

Q1. What is stemming? Why is it important in NLP?

Answer: Stemming is the process of reducing a word to its root or base form. For example, "running", "runs", and "ran" are all reduced to "run".

It is important in NLP to normalize text and group similar words together, improving text analysis and search efficiency.

Q2. What is the difference between stemming and lemmatization?

Answer:

- **Stemming** cuts off word endings without considering the meaning (e.g., "happiness" → "happi").
- **Lemmatization** returns the base word by considering context and grammar (e.g., "running" → "run").
- Lemmatization is more accurate but computationally heavier than stemming.

Q3. What are the types of stemmers available in NLTK

Answer: NLTK provides:

1. **Porter Stemmer** – Basic and widely used.
2. **Snowball Stemmer** – Advanced version of Porter with multilingual support.
3. **Lancaster Stemmer** – Very aggressive and fast, but may over-stem.

Q4. Why does "happiness" become "happi" after stemming?

Answer: Stemmers like **Porter** and **Snowball** use rule-based algorithms to remove common suffixes (-ness, -ing, -ly).

So, "happiness" → remove "ness" → "happi", even though it's not a valid English word.

Q5. Which stemmer is the most aggressive and what are its pros and cons?

Answer: **Lancaster Stemmer** is the most aggressive.

- **Pros:** Very fast, short output, simple implementation.
- **Cons:** May over-stem and distort the meaning of words. Not suitable for sensitive NLP tasks like sentiment analysis.

EXPERIMENT-12

AIM: Write a program to implement stemming for a given sentence using NLTK.

//CODE

```
import nltk
import string

nltk.download('punkt_tab', quiet=True)

from nltk.stem import (
    PorterStemmer,
    SnowballStemmer,
    LancasterStemmer
)
from nltk.tokenize import word_tokenize

class SentenceStemmer:
    def __init__(self):
        self.porter_stemmer = PorterStemmer()
        self.snowball_stemmer = SnowballStemmer('english')
        self.lancaster_stemmer = LancasterStemmer()

    def pre_process_text(self, sentence):
        # convert to lowercase
        # remove punctuation
        # tokenize words

        sentence = sentence.lower()
        sentence = sentence.translate(str.maketrans("", "", string.punctuation))
        return word_tokenize(sentence)

    def stem_sentence(self, sentence, stemmer_type='porter'):
        words = self.pre_process_text(sentence)
```

```

if stemmer_type == 'porter':
    stemmer = self.porter_stemmer
elif stemmer_type == 'snowball':
    stemmer = self.snowball_stemmer
elif stemmer_type == 'lancaster':
    stemmer = self.lancaster_stemmer
else:
    raise ValueError("Invalid stemmer type. Choose 'porter', 'snowball', or 'lancaster'")
stemmed_words = [stemmer.stem(word) for word in words]

return {
    'original_words' : words,
    'stemmed_words' : stemmed_words,
    'stemmed_sentence' : ' '.join(stemmed_words)
}

def compare_stemmers(self, sentence):
    stemmer_types = ['porter', 'snowball', 'lancaster']
    results = {}

    for stemmer_type in stemmer_types:
        results[stemmer_type] = self.stem_sentence(sentence, stemmer_type)

    return results

def main():
    stemmer = SentenceStemmer()

    sentences = [
        "Running quickly through the beautiful gardens.",
        "The programmers are developing interesting software applications.",
        "I am easily understanding complex machine learning concepts."
    ]

    for sentence in sentences:

```

```

print(f"\nOriginal Sentence: {sentence}")

comparison = stemmer.compare_stemmers(sentence)

for stemmer_type, result in comparison.items():
    print(f"\n{stemmer_type.capitalize()} Stemmer:")
    print(f"Original Words: {result['original_words']}")
    print(f"Stemmed Words: {result['stemmed_words']}")
    print(f"Stemmed Sentence: {result['stemmed_sentence']}")

if __name__ == '__main__':
    main()

```

OUTPUT

```
PS C:\Users\HP\Desktop\AI EXPERIMENTS> python exp12.py
```

Original Sentence: Running quickly through the beautiful gardens.

Porter Stemmer:

Original Words: ['running', 'quickly', 'through', 'the', 'beautiful', 'gardens']

Stemmed Words: ['run', 'quickli', 'through', 'the', 'beauti', 'garden']

Stemmed Sentence: run quickli through the beauti garden

Snowball Stemmer:

Original Words: ['running', 'quickly', 'through', 'the', 'beautiful', 'gardens']

Stemmed Words: ['run', 'quick', 'through', 'the', 'beauti', 'garden']

Stemmed Sentence: run quick through the beauti garden

Lancaster Stemmer:

Original Words: ['running', 'quickly', 'through', 'the', 'beautiful', 'gardens']

Stemmed Words: ['run', 'quick', 'through', 'the', 'beauty', 'gard']

Stemmed Sentence: run quick through the beauty gard

Original Sentence: The programmers are developing interesting software applications.

Porter Stemmer:

Original Words: ['the', 'programmers', 'are', 'developing', 'interesting', 'software', 'applications']

Stemmed Words: ['the', 'programm', 'are', 'develop', 'interest', 'softwar', 'applic']

Stemmed Sentence: the programm are develop interest softwar applic

Snowball Stemmer:

Original Words: ['the', 'programmers', 'are', 'developing', 'interesting', 'software', 'applications']

Stemmed Words: ['the', 'programm', 'are', 'develop', 'interest', 'softwar', 'applic']

Stemmed Sentence: the programm are develop interest softwar applic

Lancaster Stemmer:

Original Words: ['the', 'programmers', 'are', 'developing', 'interesting', 'software', 'applications']

Stemmed Words: ['the', 'program', 'ar', 'develop', 'interest', 'softw', 'apply']

Stemmed Sentence: the program ar develop interest softw apply

Original Sentence: I am easily understanding complex machine learning concepts.

Porter Stemmer:

Original Words: ['i', 'am', 'easily', 'understanding', 'complex', 'machine', 'learning', 'concepts']

Stemmed Words: ['i', 'am', 'easili', 'understand', 'complex', 'machin', 'learn', 'concept']

Stemmed Sentence: i am easili understand complex machin learn concept

earning', 'concepts']

earning', 'concepts']

Stemmed Words: ['i', 'am', 'easili', 'understand', 'complex', 'machin', 'learn', 'concept']

Stemmed Sentence: i am easili understand complex machin learn concept

Lancaster Stemmer:

Original Words: ['i', 'am', 'easily', 'understanding', 'complex', 'machine', 'learning', 'concepts']

Stemmed Words: ['i', 'am', 'easy', 'understand', 'complex', 'machin', 'learn', 'conceiv']

Stemmed Sentence: i am easy understand complex machin learn conceiv

PS C:\Users\HP\Desktop\AI EXPERIMENTS> █

VIVA QUESTIONS(EXP-12)

Q1. What is stemming? Why is it important in NLP?

Answer: Stemming is the process of reducing a word to its root or base form. For example, "running", "runs", and "ran" are all reduced to "run".

It is important in NLP to normalize text and group similar words together, improving text analysis and search efficiency.

Q2. What is the difference between stemming and lemmatization?

Answer:

- **Stemming** cuts off word endings without considering the meaning (e.g., "happiness" → "happi").
- **Lemmatization** returns the base word by considering context and grammar (e.g., "running" → "run").
- Lemmatization is more accurate but computationally heavier than stemming.

Q3. What are the types of stemmers available in NLTK

Answer: NLTK provides:

4. **Porter Stemmer** – Basic and widely used.
5. **Snowball Stemmer** – Advanced version of Porter with multilingual support.
6. **Lancaster Stemmer** – Very aggressive and fast, but may over-stem.

Q4. Why does "happiness" become "happi" after stemming?

Answer: Stemmers like **Porter** and **Snowball** use rule-based algorithms to remove common suffixes (-ness, -ing, -ly).

So, "happiness" → remove "ness" → "happi", even though it's not a valid English word.

Q5. Which stemmer is the most aggressive and what are its pros and cons?

Answer: **Lancaster Stemmer** is the most aggressive.

- **Pros:** Very fast, short output, simple implementation.
- **Cons:** May over-stem and distort the meaning of words. Not suitable for sensitive NLP tasks like sentiment analysis.

EXPERIMENT-13

AIM: Write a program to POS (Part of Speech) tagging for the given sentence.

//CODE

```
import nltk

from nltk import word_tokenize

import pandas as pd


# Download necessary NLTK data

# This is required for first-time use

nltk.download('punkt')

nltk.download('averaged_perceptron_tagger')


def pos_tag_sentence(sentence):

    # Tokenize the sentence into words

    words = word_tokenize(sentence)


    # Perform POS tagging

    pos_tags = nltk.pos_tag(words)


    # Create a DataFrame for better visualization of results

    df = pd.DataFrame(pos_tags, columns=['Word', 'POS Tag'])


    # Print original sentence

    print(f"Original Sentence: {sentence}\n")


    # Print POS tags
```

```
print("POS Tagging Results:")
```

```
print(df)
```

```
# Explain some common POS tags
```

```
common_tags = {
```

```
    'NN': 'Noun, singular',
```

```
    'NNS': 'Noun, plural',
```

```
    'NNP': 'Proper noun, singular',
```

```
    'NNPS': 'Proper noun, plural',
```

```
    'VB': 'Verb, base form',
```

```
    'VBD': 'Verb, past tense',
```

```
    'VBG': 'Verb, gerund/present participle',
```

```
    'VBN': 'Verb, past participle',
```

```
    'VBP': 'Verb, non-3rd person singular present',
```

```
    'VBZ': 'Verb, 3rd person singular present',
```

```
    'JJ': 'Adjective',
```

```
    'JJR': 'Adjective, comparative',
```

```
    'JJS': 'Adjective, superlative',
```

```
    'RB': 'Adverb',
```

```
    'DT': 'Determiner',
```

```
    'IN': 'Preposition or subordinating conjunction',
```

```
    'PRP': 'Personal pronoun',
```

```
    'CC': 'Coordinating conjunction'
```

```
}
```

```
print("\nCommon POS Tag Explanations:")
```

```
# Get unique tags from our results
```

```
unique_tags = set([tag for _, tag in pos_tags])
```

```
# Print explanations for tags that appear in our sentence
```

```
for tag in unique_tags:
```

```
    if tag in common_tags:
```

```
        print(f"{tag}: {common_tags[tag]}")
```

```
return pos_tags
```

```
# Example usage
```

```
if __name__ == "__main__":
```

```
# Test with multiple sentences
```

```
sentences = [
```

```
    "NLTK is a powerful Python library for natural language processing.",
```

```
    "The quick brown fox jumps over the lazy dog.",
```

```
    "I am studying machine learning and artificial intelligence at the university."
```

```
]
```

```
for i, sentence in enumerate(sentences):
```

```
    pos_tags = pos_tag_sentence(sentence)
```

```
    if i < len(sentences) - 1: # Don't print separator after the last sentence
```

```
        print("\n" + "="*50 + "\n")
```

OUTPUT

```
[nltk_data] Downloading package punkt to /Users/username/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]   /Users/username/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]   date!
```

Original Sentence: NLTK is a powerful Python library for natural language processing.

POS Tagging Results:

	Word	POS	Tag
0	NLTK	NNP	
1	is	VBZ	
2	a	DT	
3	powerful	JJ	
4	Python	NNP	
5	library	NN	
6	for	IN	
7	natural	JJ	
8	language	NN	
9	processing	NN	
10	.	.	

Common POS Tag Explanations:

NNP: Proper noun, singular

VBZ: Verb, 3rd person singular present

DT: Determiner

JJ: Adjective

NN: Noun, singular

IN: Preposition or subordinating conjunction

=====

Original Sentence: The quick brown fox jumps over the lazy dog.

POS Tagging Results:

	Word	POS	Tag
0	The	DT	
1	quick	JJ	
2	brown	JJ	
3	fox	NN	
4	jumps	VBZ	
5	over	IN	
6	the	DT	
7	lazy	JJ	
8	dog	NN	
9	.	.	

Common POS Tag Explanations:

DT: Determiner

JJ: Adjective

NN: Noun, singular

VBZ: Verb, 3rd person singular present

IN: Preposition or subordinating conjunction

=====

Original Sentence: I am studying machine learning and artificial intelligence at the university.

POS Tagging Results:

	Word	POS	Tag
0	I	PRP	
1	am	VBP	
2	studying	VBG	
3	machine	NN	
4	learning	NN	
5	and	CC	
6	artificial	JJ	
7	intelligence	NN	
8	at	IN	
9	the	DT	
10	university	NN	
11	.	.	

Common POS Tag Explanations:

PRP: Personal pronoun

VBP: Verb, non-3rd person singular present

VBG: Verb, gerund/present participle

NN: Noun, singular

CC: Coordinating conjunction

JJ: Adjective

IN: Preposition or subordinating conjunction

DT: Determiner

VIVA QUESTIONS(EXP-13)

1. What is POS tagging and why is it important in Natural Language Processing?

POS (Part of Speech) tagging is the process of marking words in text with their corresponding grammatical categories (noun, verb, adjective, etc.) based on definition and context. It's vital in NLP because it provides fundamental grammatical information needed for higher-level tasks like parsing, information extraction, and machine translation. POS tags help resolve word sense ambiguities and establish relationships between words in sentences, forming the foundation for more complex language understanding tasks.

2. Explain the difference between rule-based and statistical POS tagging approaches.

Rule-based POS tagging employs manually crafted linguistic rules and dictionaries to assign tags based on grammatical patterns. These systems are interpretable but struggle with exceptions and new patterns. Statistical approaches learn tag probabilities from pre-annotated corpora, making decisions based on contextual patterns rather than explicit rules. While rule-based systems leverage linguistic expertise, statistical methods handle novel text better but require substantial training data and may lack explainability.

3. What challenges might you encounter when performing POS tagging on social media text?

Social media text presents unique challenges for POS tagging including non-standard spellings, abbreviations, and slang that aren't in typical training data. Users often ignore conventional grammar and punctuation, creating fragmented sentences lacking structural cues. The presence of hashtags, emojis, and frequent code-switching (mixing languages) further complicates tagging. Additionally, internet language evolves rapidly, requiring taggers to continuously adapt to new terms and usages.

4. How does NLTK's POS tagger work, and what tagset does it use?

NLTK's default POS tagger uses the Perceptron algorithm to learn from pre-tagged text, analyzing features like surrounding words, prefixes, suffixes, and capitalization to predict tags. It employs the Penn Treebank tagset, which contains about 36 distinct tags that differentiate between various verb forms, noun types, and other grammatical categories. For unknown words, the tagger relies on morphological features and contextual patterns to make educated guesses about the appropriate classification.

5. How can the accuracy of POS tagging be evaluated, and what metrics are commonly used?

POS tagging accuracy is evaluated by comparing system output against a manually annotated gold standard corpus. The primary metric is overall accuracy (percentage of correctly tagged tokens), while per-tag precision, recall, and F1-scores provide more detailed analysis of performance across different parts of speech. Cross-validation ensures robust evaluation, and confusion matrices help identify systematic errors between frequently confused tag pairs. Domain-specific testing is important since tagger performance often varies across different text genres.

EXPERIMENT-14

AIM: Write a program to implement Lemmatization using NLTK.

//CODE

```
import nltk

from nltk.stem import WordNetLemmatizer

from nltk import word_tokenize, pos_tag

import pandas as pd


# Download necessary NLTK data

# Uncomment these lines if running for the first time

nltk.download('punkt')

nltk.download('wordnet')

nltk.download('averaged_perceptron_tagger')


def get_wordnet_pos(tag):

    """

    Map POS tag to first character used by WordNet lemmatizer

    """

    tag_dict = {

        'J': nltk.corpus.wordnet.ADJ,

        'N': nltk.corpus.wordnet.NOUN,

        'V': nltk.corpus.wordnet.VERB,

        'R': nltk.corpus.wordnet.ADV

    }

    return tag_dict.get(tag[0], nltk.corpus.wordnet.NOUN)


def lemmatize_text(text):

    """

    Lemmatize text using NLTK's WordNetLemmatizer

    """

    lemmatizer = WordNetLemmatizer()
```

```

# Tokenize the text
tokens = word_tokenize(text)

# Get POS tags
pos_tags = pos_tag(tokens)

# Lemmatize each word based on POS tag
lemmatized_words = []
for word, tag in pos_tags:
    wordnet_pos = get_wordnet_pos(tag)
    lemmatized_word = lemmatizer.lemmatize(word, wordnet_pos)
    lemmatized_words.append((word, tag, lemmatized_word))

# Create DataFrame for display
df = pd.DataFrame(lemmatized_words, columns=['Original Word', 'POS Tag', 'Lemmatized Word'])

# Print results
print(f"Original Text: {text}\n")
print("Lemmatization Results:")
print(df)
print()

# Show the lemmatized text
lemmatized_text = ' '.join([word for _, _, word in lemmatized_words])
print(f"Lemmatized Text: {lemmatized_text}")

return lemmatized_words

# Example usage
if __name__ == "__main__":
    print("="*60)
    print("NLTK LEMMATIZATION DEMONSTRATION")
    print("="*60 + "\n")

```

```
# Test with multiple examples
```

```
examples = [
```

```
    "The cats are running and jumping in the gardens",
```

```
    "I am studying studies studied by the best students",
```

```
    "Better algorithms running faster than the fastest computers",
```

```
    "He was walking to the store while eating an apple"
```

```
]
```

```
for i, example in enumerate(examples, 1):
```

```
    print(f'Example {i}:')
```

```
    lemmatize_text(example)
```

```
    if i < len(examples):
```

```
        print("\n" + "-"*60 + "\n")
```

OUTPUT

```
=====
NLTK LEMMATIZATION DEMONSTRATION
=====

[nltk_data] Downloading package punkt to /Users/username/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]   /Users/username/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]   /Users/username/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
```

Example 1:

Original Text: The cats are running and jumping in the gardens

Lemmatization Results:

	Original Word	POS Tag	Lemmatized Word
0	The	DT	The
1	cats	NNS	cat
2	are	VBP	be
3	running	VBG	run
4	and	CC	and
5	jumping	VBG	jump
6	in	IN	in
7	the	DT	the
8	gardens	NNS	garden

Lemmatized Text: The cat be run and jump in the garden

Example 2:

Original Text: I am studying studies studied by the best students

Lemmatization Results:

	Original Word	POS Tag	Lemmatized Word
0	I	PRP	I
1	am	VBP	be
2	studying	VBG	study
3	studies	NNS	study
4	studied	VCN	study
5	by	IN	by
6	the	DT	the
7	best	JJ	best
8	students	NNS	student

Lemmatized Text: I be study study study by the best student

Example 3:

Original Text: Better algorithms running faster than the fastest computers

Lemmatization Results:

	Original Word	POS Tag	Lemmatized Word
0	Better	JJR	better
1	algorithms	NNS	algorithm
2	running	VBG	run
3	faster	JJR	faster
4	than	IN	than
5	the	DT	the
6	fastest	JJS	fastest
7	computers	NNS	computer

Lemmatized Text: better algorithm run faster than the fastest computer

Example 4:

Original Text: He was walking to the store while eating an apple

Lemmatization Results:

	Original Word	POS Tag	Lemmatized Word
0	He	PRP	He
1	was	VBD	be
2	walking	VBG	walk
3	to	TO	to
4	the	DT	the
5	store	NN	store
6	while	IN	while
7	eating	VBG	eat
8	an	DT	an
9	apple	NN	apple

Lemmatized Text: He be walk to the store while eat an apple

VIVA QUESTIONS(EXP-14)

1. What is lemmatization and how does it differ from stemming?

Lemmatization is the process of reducing words to their base or dictionary form (lemma) while considering the context and part of speech. Unlike stemming, which simply chops off word endings based on rules, lemmatization ensures the resulting form is an actual dictionary word. For example, stemming might reduce "better" to "bett," while lemmatization correctly produces "good." Lemmatization is computationally more intensive than stemming but provides more accurate results, particularly for irregular forms like "is/was/been" → "be" or "mice" → "mouse."

2. Why is POS tagging important for effective lemmatization?

POS (Part of Speech) tagging is crucial for effective lemmatization because many words have different lemmas depending on their grammatical function. For example, the word "leaves" would lemmatize to "leaf" when used as a noun but to "leave" when used as a verb. Without POS information, a lemmatizer would have to guess which base form is appropriate. By providing part-of-speech context, POS tagging allows the lemmatizer to select the correct transformation rule, significantly improving accuracy for ambiguous words and ensuring appropriate lemmas across different grammatical categories.

3. What are some applications of lemmatization in NLP systems?

Lemmatization is widely used in information retrieval systems to match variations of the same word in queries and documents, improving search accuracy. In text classification and topic modeling, it helps reduce vocabulary size by consolidating word variations, leading to more robust feature representations. For sentiment analysis, lemmatization ensures that different forms of the same word contribute to the same sentiment score. Machine translation systems use lemmatization to help identify word equivalents across languages. Additionally, lemmatization assists in document summarization by helping identify key concepts regardless of their various word forms.

4. How does WordNet support lemmatization in NLTK?

WordNet in NLTK supports lemmatization by providing a comprehensive lexical database that maps words to their canonical forms based on part of speech. The WordNetLemmatizer accesses this database to look up the appropriate lemma for each word according to its grammatical category. This approach handles irregular forms that rule-based systems would struggle with, such as "feet" → "foot" or "better" → "good." WordNet also includes hierarchical relationships between words, enabling the lemmatizer to understand word semantics and not just morphology, which is particularly helpful for disambiguating words with multiple potential base forms.

EXPERIMENT-15

AIM: Write a program to for Text Classification for a given sentence using NLTK.

//CODE

```
import nltk

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.classify import NaiveBayesClassifier
from nltk.classify.util import accuracy
import random


# Download necessary NLTK data
nltk.download('punkt')
nltk.download('stopwords')


def preprocess_text(text):
    """
    Preprocess text by tokenizing and removing stopwords
    """
    # Tokenize
    tokens = word_tokenize(text.lower())

    # Remove stopwords
    stop_words = set(stopwords.words('english'))
    filtered_tokens = [token for token in tokens if token.isalpha() and token not in stop_words]

    return filtered_tokens


def extract_features(words):
    """
    Create a feature dictionary from words
    """
    return dict([(word, True) for word in words])
```

```

def train_classifier():

    # Sample dataset - positive and negative sentences
    positive_sentences = [
        "I love this product, it's amazing!",
        "The service was excellent and staff very friendly",
        "This movie was fantastic and entertaining",
        "I had a great experience with this company",
        "The food was delicious and well presented",
        "I'm very happy with my purchase",
        "The hotel room was clean and comfortable",
        "That was the best concert I've ever attended",
        "The customer support team was very helpful",
        "This book is brilliant and insightful"
    ]

    negative_sentences = [
        "This product is terrible and broke quickly",
        "The service was slow and the staff was rude",
        "I didn't enjoy this movie at all",
        "I had a terrible experience with this company",
        "The food was cold and tasteless",
        "I regret buying this item",
        "The hotel room was dirty and noisy",
        "That was the worst concert I've ever attended",
        "The customer support was unhelpful",
        "This book is boring and poorly written"
    ]

    # Process the datasets
    positive_features = [(extract_features(preprocess_text(sentence)), 'positive')
                        for sentence in positive_sentences]

    negative_features = [(extract_features(preprocess_text(sentence)), 'negative')

```

```
        for sentence in negative_sentences]
```

```
# Combine and shuffle the datasets
```

```
all_features = positive_features + negative_features
```

```
random.shuffle(all_features)
```

```
# Split into training (80%) and testing (20%)
```

```
split_point = int(len(all_features) * 0.8)
```

```
train_set = all_features[:split_point]
```

```
test_set = all_features[split_point:]
```

```
# Train a Naive Bayes classifier
```

```
classifier = NaiveBayesClassifier.train(train_set)
```

```
# Print classifier accuracy
```

```
print(f'Classifier accuracy: {accuracy(classifier, test_set):.2f}')
```

```
# Show most informative features
```

```
print("\nMost Informative Features:")
```

```
classifier.show_most_informative_features(5)
```

```
return classifier
```

```
def classify_text(classifier, text):
```

```
    """
```

```
    Classify a text as positive or negative
```

```
    """
```

```
    processed_text = preprocess_text(text)
```

```
    features = extract_features(processed_text)
```

```
# Get the classification result
```

```
classification = classifier.classify(features)
```

```
# Get probability distribution
```

```

prob_dist = classifier.prob_classify(features)
positive_prob = round(prob_dist.prob('positive') * 100, 2)
negative_prob = round(prob_dist.prob('negative') * 100, 2)

return classification, positive_prob, negative_prob, processed_text

```

```

def main():
    print("="*70)
    print("          TEXT SENTIMENT CLASSIFICATION WITH NLTK")
    print("="*70)

    print("\nTraining classifier with sample data...")
    classifier = train_classifier()

    print("\n" + "="*70)
    print("          CLASSIFICATION RESULTS")
    print("="*70)

    # Test sentences to classify
    test_sentences = [
        "I really enjoyed the new restaurant downtown",
        "The customer service was terrible and unprofessional",
        "This phone has amazing features and battery life",
        "I waited two hours and the food was cold when it arrived",
        "The performance was neither good nor bad, just average"
    ]

    for i, sentence in enumerate(test_sentences, 1):
        classification, pos_prob, neg_prob, tokens = classify_text(classifier, sentence)

        print(f"\nTest Example {i}:")
        print(f"Text: \"{sentence}\"")
        print(f"Processed tokens: {tokens}")
        print(f"Classification: {classification.upper()}")

```



```

print(f'Confidence: Positive ({pos_prob}%), Negative ({neg_prob}%)\n')
print("-"*70)

if __name__ == "__main__":
    main()

```

OUTPUT

```

PS C:\Users\HP\Desktop\AI EXPERIMENTS> python exp15.py
[nltk_data] Downloading package punkt to
[nltk_data]   C:\Users\HP\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]   C:\Users\HP\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
=====
                        TEXT SENTIMENT CLASSIFICATION WITH NLTK
=====

Training classifier with sample data...
Classifier accuracy: 0.00

Most Informative Features:
Most Informative Features
          amazing = None          negati : positi =      1.1 : 1.0
          attended = None         positi : negati =      1.1 : 1.0
          boring = None          positi : negati =      1.1 : 1.0
        brilliant = None         negati : positi =      1.1 : 1.0
          broke = None           positi : negati =      1.1 : 1.0

```

```
=====
CLASSIFICATION RESULTS
=====
```

Test Example 1:

Text: "I really enjoyed the new restaurant downtown"

Processed tokens: ['really', 'enjoyed', 'new', 'restaurant', 'downtown']

Classification: POSITIVE

Confidence: Positive (50.0%), Negative (50.0%)

Test Example 2:

Text: "The customer service was terrible and unprofessional"

Processed tokens: ['customer', 'service', 'terrible', 'unprofessional']

Classification: NEGATIVE

Confidence: Positive (25.0%), Negative (75.0%)

Test Example 3:

Text: "This phone has amazing features and battery life"

Processed tokens: ['phone', 'amazing', 'features', 'battery', 'life']

Classification: POSITIVE

Confidence: Positive (75.0%), Negative (25.0%)

Test Example 4:

Text: "I waited two hours and the food was cold when it arrived"

Processed tokens: ['waited', 'two', 'hours', 'food', 'cold', 'arrived']

Classification: NEGATIVE

Confidence: Positive (25.0%), Negative (75.0%)

Test Example 5:

Text: "The performance was neither good nor bad, just average"

Processed tokens: ['performance', 'neither', 'good', 'bad', 'average']

Classification: POSITIVE

Confidence: Positive (50.0%), Negative (50.0%)

VIVA QUESTIONS(EXP-15)

Q1: What is text classification and what are its common applications?

A: Text classification is the process of categorizing text documents into predefined classes or categories based on their content. Common applications include sentiment analysis (determining if text is positive/negative/neutral), spam detection, topic categorization, intent detection in chatbots, language identification, and content moderation. It's widely used in customer feedback analysis, social media monitoring, and automated content organization.

Q2: Explain the significance of preprocessing in NLTK text classification.

A: Preprocessing enhances classification accuracy by standardizing text data. It includes tokenization (breaking text into words), removing stopwords (common words like "the" that don't carry meaning), stemming/lemmatization (reducing words to root forms), and case normalization. These steps reduce dimensionality, remove noise, and highlight important features. Without preprocessing, irrelevant text patterns could confuse the classifier and lead to poor performance.

Q3: What is TF-IDF vectorization and why is it used in text classification?

A: TF-IDF (Term Frequency-Inverse Document Frequency) is a numerical statistic that reflects the importance of a word in a document relative to a collection of documents. It combines how frequently a term appears in a document (TF) with how rare it is across all documents (IDF). It's used in text classification to convert text to vectors while giving higher weight to distinctive terms and lower weight to common terms, thus improving classification by focusing on significant words.

Q4: How does Naive Bayes classification work for text analysis?

A: Naive Bayes classifiers are probabilistic algorithms based on Bayes' theorem with an assumption of feature independence. For text, it calculates the probability of a document belonging to a class based on word occurrences. Despite its "naive" assumption that words appear independently, it performs well for text classification because it can handle high-dimensional data efficiently, requires relatively small training data, and is computationally inexpensive, making it ideal for initial text classification tasks.

Q5: What metrics are important for evaluating text classification models?

A: Key metrics include precision (percentage of correct positive predictions), recall (percentage of actual positives identified), F1-score (harmonic mean of precision and recall), and accuracy (overall correct predictions). For imbalanced datasets, precision, recall, and F1-score are more informative than accuracy. Additionally, the confusion matrix provides visual insight into classification performance by showing true/false positives and negatives, helping identify specific classification errors.